
TransactionGPT

Visa Research

Abstract

We present **TransactionGPT** (TGPT), a foundation model for consumer transaction data within one of the world’s largest payment networks. TGPT is designed to understand and generate transaction trajectories while simultaneously supporting a variety of downstream prediction and classification tasks. We introduce a novel 3D-Transformer architecture specifically tailored for capturing the complex dynamics in payment transaction data. This architecture incorporates design innovations that enhance modality fusion and computational efficiency, while seamlessly enabling joint optimization with downstream objectives. Trained on billion-scale real-world transactions, TGPT significantly improves downstream anomaly transaction detection performance against a competitive production model and exhibits advantages over baselines in generating future transactions. We conduct extensive empirical evaluations utilizing a diverse collection of company transaction datasets spanning multiple downstream tasks, thereby enabling a thorough assessment of TGPT’s effectiveness and efficiency in comparison to established methodologies. Furthermore, we examine the incorporation of LLM-derived embeddings within TGPT and benchmark its performance against fine-tuned LLMs, demonstrating that TGPT achieves superior predictive accuracy as well as faster training and inference. We anticipate that the architectural innovations and practical guidelines from this work will advance foundation models for transaction-like data and catalyze future research in this emerging field.

1 Introduction

Foundation models have emerged as a transformative paradigm in artificial intelligence, characterized by their ability to acquire broad capabilities from self-supervised training on large-scale data and to adapt effectively to a wide range of downstream tasks [Bommasani, 2021]. By encoding rich prior knowledge, these models serve as a common backbone across domains from natural language processing [Radford et al., 2019] to computer vision [Dosovitskiy et al., 2020] and even specialized fields such as the medical industry Moor et al. [2023]. A prominent and highly successful class of foundation models is built upon the Transformer architecture [Vaswani et al., 2017], whose self-attention mechanism excels at modeling intricate dependencies between tokens in sequential and structured data. Scaling Transformer-based models to billions of parameters in accordance with empirical scaling laws [Kaplan et al., 2020] has yielded powerful systems that now underpin state-of-the-art language and vision applications.

Despite the widespread adoption of foundation models, only a few endeavors [Skalski et al., 2023, Brüel Gabriellsson et al., 2025, Li et al., 2025, Aminian et al., 2025] have explored their application to payment data, and others only disclose vague or incomplete information [Stripe, 2025]. While limited access to real-world payment network data may contribute to this gap, the primary obstacle is the large-scale and complex structural nature of **Multi-Modal-Temporal-Tabular (MMTT)** transaction data. As shown in Fig. 1, a temporal transaction sequence from the same consumer often comprises transactions from multiple sources; meanwhile, each transaction has diverse multi-modality fields including metadata and downstream features (more details in Sec. 2.1). Transformer-based architec-

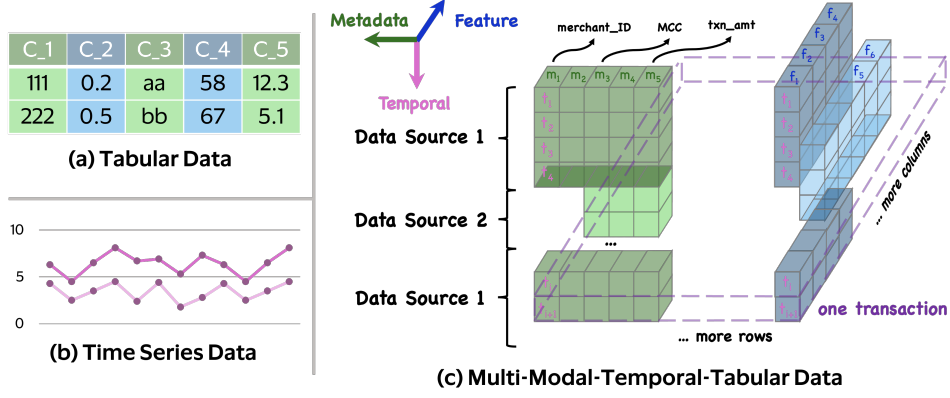


Figure 1: A comparison of multi-modal temporal tabular payments data and other similar data types.

tures provide a natural framework for encoding transaction sequence data. Just as language models predict future words based on preceding context, we posit that consumer transaction trajectories can be forecast using historical transactions.

We propose to learn invariant patterns in individual and collective shopping behavior through foundation model training on large-scale transaction data. However, applying existing Transformer architectures to encode transaction sequences is non-trivial. **First**, payment transaction data possesses domain-specific characteristics, with predominantly numerical and categorical fields that have limited semantic richness [Bahnsen et al., 2016]. This makes direct LLM adoption both computationally costly and ineffective, as their linguistic capabilities provide minimal value for transaction data comprehension. **Second**, transaction data are stored as database tables. Existing tabular foundation models [Hollmann et al., 2025, Hegselmann et al., 2023], however, are inadequate for this domain as they either handle only small-scale data or require semantically rich column names. Furthermore, most are tailored for classification tasks and lack the flexibility to support the varied downstream applications essential for industrial payment systems. **Third**, although transaction sequences bear superficial resemblance to time series [Wen et al., 2023] or point process [Mei and Eisner, 2017] data, they differ fundamentally in data complexity: time series or point process consist of single or multiple scalar values per timestep, while transactions contain multi- and high-dimensional information, including metadata with various cardinalities and derived features. This structural difference makes existing temporal data foundation models [Garza et al., 2023, Ekambaram et al., 2024] inadequate for transaction sequence modeling. We argue that a foundation model for the consumer payment industry should accommodate: 1) the complex, heterogeneous nature of MMTT payment data; 2) diverse requirements across downstream applications; 3) stringent efficiency and latency constraints.

To overcome these challenges and develop a foundation model tailored to the consumer payment industry, this paper presents **Transaction Generative Predictive Transformer (TransactionGPT or TGPT)**, a specialized foundation model for transaction-like MMTT data, with direct relevance and broad applicability for the consumer payment industry. TransactionGPT is a Transformer-based model trained on billions of transactions to address various downstream tasks in payment, including prediction, classification, and representation learning. The model features a novel 3-Dimensional (3D) architecture that utilizes three specialized Transformers to hierarchically encode transaction features, individual transaction representations, and sequential transaction patterns. Moreover, we introduce a novel *virtual token mechanism* that enables effective modality fusion between different architectural dimensions. This design dramatically reduces the computational complexity of the 3D-Transformer structure while boosting performance across downstream applications. Notably, the TransactionGPT architecture maintains high flexibility through modular, switchable components at each dimension and layer, facilitating the integration of emerging Transformer and generative modeling innovations.

We highlight our contributions as follows:

- We introduce TransactionGPT, a foundation model for understanding and generating consumer transaction trajectories, trained on billion-scale data from one of the world’s largest payment networks. It includes a 3D-Transformer for encoding complex transaction data and a virtual token mechanism to enable effective cross-modality information fusion.
- We present a series of techniques developed to ensure the scalability and efficiency of TransactionGPT for real-world deployment and quantify their performance impact. We also provide a detailed analysis of our architectural design choices, offering insights and lessons learned from the model’s development and evaluation to benefit the practitioners in the community.
- We evaluate TGPT under three payment-related tasks to demonstrate its foundational capability in transaction anomaly detection, generation, and representation learning. The results show that TransactionGPT achieves a 22% relative improvement over the production model on a business-critical anomaly detection metric and demonstrates superior efficiency compared to LLM-based foundation models.

2 Preliminary

In this section, we characterize the complex and hierarchical structure of transaction data, establish the notations used throughout the paper, and formally define our problem.

2.1 Multi-Modal-Temporal-Tabular Data

Temporal-tabular structure. The transaction sequence from a single payment account is given in a temporal-tabular format, denoted by $\mathcal{S} = [\mathbf{tr}_1, \mathbf{tr}_2, \dots, \mathbf{tr}_n]$, where $\mathbf{tr}_1$ is the earliest transaction and $\mathbf{tr}_n$ is the latest. Each transaction $\mathbf{tr}$ is extracted from the transaction table and represented as a d -dimensional vector comprising the following components in tabular form:

$$\mathbf{tr} = [\mathcal{M} \oplus \mathcal{E} \oplus \mathcal{F}] \in \mathbb{R}^d, \quad (1)$$

where $\mathcal{M}$ denotes the vector of metadata associated with $\mathbf{tr}$, including essential numerical attributes such as the transaction amount and timestamp. $\mathcal{E}$ represents the one-hot entity vectors linked to the current transaction. Common entities include merchant ID and merchant category. Entities are part of metadata in a broader sense; we use a distinct symbol $\mathcal{E}$ because they are encoded differently from other categorical metadata (see Sec. 3.2.3). Meanwhile, the embedding of $\mathcal{E}$ can be dynamically obtained as the output of another model or precomputed and retrieved from a vector database. Metadata $\mathcal{M}$ and entities $\mathcal{E}$ are present in every transaction and are independent of any downstream task. Unless otherwise specified, the term *metadata* in this paper refers to $\mathcal{M} \oplus \mathcal{E}$. Additionally, $\mathcal{F}$ is an optional feature vector based on the availability of downstream task-dependent features. $\mathcal{F}$ usually consists of hundreds to thousands of numerical and categorical values, depending on the specific requirements of the downstream task.

Multi-modal structure. The metadata fields $\mathcal{M} \oplus \mathcal{E}$ (typically totaling 10 to 20 fields, mostly categorical) within the same $\mathbf{tr}$ collectively describe the basic property of $\mathbf{tr}$ and are highly correlated with each other. For example, the merchant category, merchant name, location, and price are all interdependent when purchasing a pair of shoes. Moreover, some metadata fields exhibit an extremely high cardinality, e.g., merchant ID may have millions of unique values, necessitating large embedding sizes during data encoding. In contrast, the feature vector $\mathcal{F}$ does not contain explicit transaction metadata but includes features specifically crafted for a downstream task. $\mathcal{F}$ often consists of hundreds or thousands of scalar values (mostly numerical), each with low cardinality and thus small embedding sizes. Accordingly, metadata and feature vectors effectively constitute *two distinct data modalities*, which are embedded and modeled using different strategies within TGPT. Consequently, transaction data exhibits a multi-modal-temporal-tabular (**MMTT**) structure in general, as shown in Fig. 1.

2.2 Problem Definition

Analogous to how LLMs are pretrained to generate future text tokens [Radford et al., 2019, Touvron et al., 2023], TGPT is trained on MMTT transaction data to generate future transaction trajectories and predict transaction properties. We formally define the problem as follows:

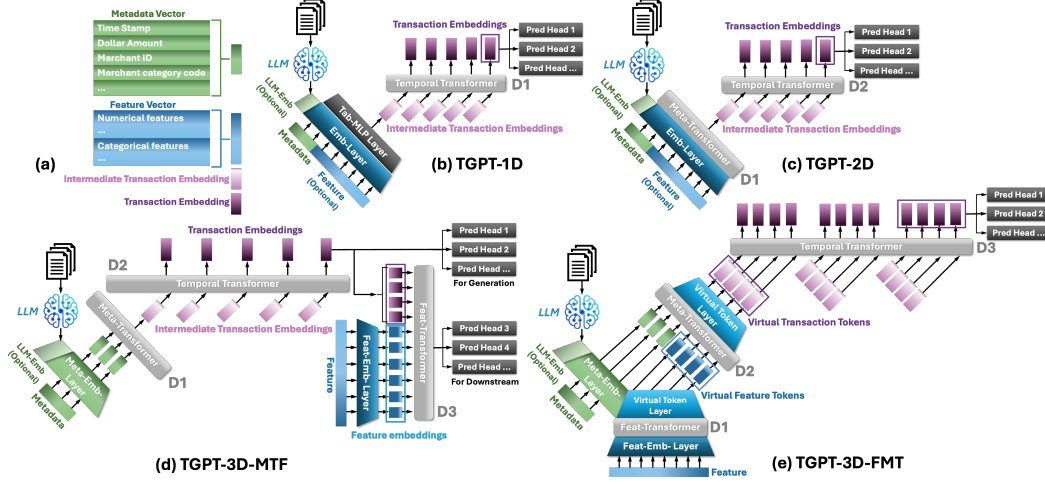


Figure 2: TransactionGPT (TGPT) model architecture evolution. (a) Diagram of the metadata vector, feature vector, intermediate transaction embedding, and final transaction embedding. (b) TGPT-1D. (c) TGPT-2D. (d) TGPT-3D-MTF (Metadata-Temporal-Feature). (e) TGPT-3D-FMT (Feature-Metadata-Temporal).

Definition 1. Given a set of MMTT transaction sequences $\{\mathcal{S}\}, \mathcal{S} \in \mathcal{S}_{train}$, our goal is to train a Transformer-based model f (i.e., TGPT) to encode the transaction information and learn the temporal dynamics at scale. Given the historical transaction sequences $\mathcal{S}_{test} = [\mathbf{tr}_1, \mathbf{tr}_2, \dots, \mathbf{tr}_n]$, an optimal TransactionGPT should be able to generate the next t transactions $[\mathbf{tr}_{n+1}, \mathbf{tr}_{n+2}, \dots, \mathbf{tr}_{n+t}]$. For each generated transaction $\mathbf{tr}_{n+i} (1 \leq i \leq t)$, the model can also produce a predicted label $\mathbf{L}_{n+i}$ if required by a downstream classification task.

With the data and problem formally defined, we now turn to the design of a modeling framework capable of effectively capturing the complex characteristics of MMTT transaction data. The following section first summarizes the architectural evolution of TGPT and presents the technical details of each model component.

3 Methodology

In this section, we first summarize the evolution path of the TGPT model from 1- to 3-Dimensional and innovations in the fusion of modality between Transformers in different dimensions (Sec. 3.1). Then we introduce the technical details of Transformer in each dimension (Sec. 3.2) and our novel modality fusion method (Sec. 3.3), followed by model time complexity analysis (Sec. 3.4) and optimization (Sec. 3.5).

3.1 Model Architecture Evolution

During the process of acquiring deeper insights into MMTT data and designing more effective model components to capture the intrinsic patterns of consumer payment trajectories, the architecture of TransactionGPT has undergone a gradual evolution. Initially, we employ a straightforward single temporal Transformer (1D), which is subsequently advanced to a structure comprising two inter-coupled Transformers for metadata and temporal modality encoding (2D), and ultimately to three inter-coupled transformers (3D) with an additional Transformer to encode features. Each stage introduces necessary sophistication to better model the underlying data characteristics, as shown in Fig. 2. Each additional Transformer—corresponding to an added dimension (data modality) in the architecture—is introduced to more effectively encode information from its respective data modality.

TGPT-1D. Only a single Transformer is used to model the temporal dependencies between transactions along the payment trajectory $\mathcal{S}$, as shown in Fig. 2(b). Meanwhile, the interaction among the various fields within the same transaction is captured by a tabular MLP block. Sec. 3.2.1 has more details about the temporal Transformer $\text{TF}(\mathcal{S})$.

Table 1: The computation formulations of the embedding of a transaction $\mathbf{tr}$ given $\mathcal{M}, \mathcal{E}, \mathcal{F} \in \mathbf{tr}$. Operation Integrate aggregates a 2D tensor (a set of embedding vectors) into a 1D tensor (a single embedding vector). Its design choices are discussed in Sec. 6.

Model	Transaction Embedding Formulation	Structure
TGPT-1D	$E^{\mathbf{tr}} = \text{MLP}(\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E}) \oplus \text{Emb}(\mathcal{F}))$	1D: $1 \times d_{\mathbf{tr}}$
TGPT-2D	$E^{\mathbf{tr}} = \text{Integrate}(\text{TFEncoder}(\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E}) \oplus \text{Emb}(\mathcal{F})))$, see Eq. 8	1D: $1 \times d_{\mathbf{tr}}$
TGPT-3D-MTF	$E^{\mathbf{tr}} = \text{Integrate}(\text{TFEncoder}(\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E})))$, see Eq. 5	1D: $1 \times d_{\mathbf{tr}}$
TGPT-3D-FMT	$E^{\mathbf{tr}} = \text{VTL}(\text{TFEncoder}(\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E}) \oplus \text{VTL}(\text{TFEncoder}(\mathcal{F}))))$, see Eq. 10	2D: $v_t \times \frac{d_{\mathbf{tr}}}{v_t}$

TGPT-2D. To more effectively encode the interactions between different data fields within one transaction $\mathbf{tr}$ (Eq. 1), an additional Transformer is incorporated, replacing the tabular MLP, resulting in a 2D transformer architecture, as shown in Fig. 2(c). In Sec. 3.2.3, we present how we process and encode different transaction metadata $\mathcal{M}$ to facilitate their interaction through the metadata Transformer $\text{TF}(\mathbf{tr})$.

TGPT-3D. As we adopt TGPT towards downstream tasks with domain specific features, we think a separate feature transformer $\text{TF}(\mathcal{F})$ is necessary. The reason has two folds: **First**, the feature set $\mathcal{F}$ is way larger than the metadata and entity set ($|\mathcal{F}| \gg |\mathcal{M}| + |\mathcal{E}|$); features within $\mathcal{F}$ are highly correlated, but their semantics are only related to the corresponding downstream task [Bahnsen et al., 2016]. **Second**, metadata and downstream features have conflicting embedding requirements: high-cardinality fields like merchant ID with millions of unique values need large embedding sizes to capture their rich information, while downstream features can be effectively represented with much smaller embeddings. In the 2D Transformer, all fields must share the same embedding size, creating an impossible trade-off where large embeddings cause excessive computational costs and overfitting risk for downstream features, while small embeddings fail to provide sufficient expressive capacity for high-cardinality metadata fields. Therefore, encoding features and metadata with distinct Transformers allows each to use optimally-sized embeddings tailored to their specific requirements, resulting in a 3D Transformer architecture, as shown in Fig. 2(d) and (e). Sec. 3.2.2 has extended details about the feature Transformer $\text{TF}(\mathcal{F})$.

Modality Fusion. Once the three Transformers are designated to encode and generate embeddings for the temporal, metadata, and feature modalities, the challenge of designing an effective modality embedding fusion mechanism becomes both critical and complex. Specifically, the small-sized embedding output of $\text{TF}(\mathcal{F})$ need to be merged with the large-sized embeddings of metadata fields into a unified representation that encapsulates all relevant information for a single transaction $\mathbf{tr}$, which can be efficiently processed by $\text{TF}(\mathbf{tr})$ and $\text{TF}(\mathcal{S})$. A naive approach, concatenating all feature and metadata embeddings into a single vector and feeding it directly into $\text{TF}(\mathcal{S})$, is costly in terms of parameter count and processing time. Extensive experiments also reveal that employing an MLP to rescale the concatenated embeddings to a manageable size for $\text{TF}(\mathcal{S})$ leads to suboptimal performance (see Sec. 5.1). The above compression approach introduces an information bottleneck: the small embedding size restricts information flow, while embeddings large enough for adequate information preservation dramatically increase computational costs. To address the challenge, we propose a novel **virtual token** based 2-step modality fusion mechanism. This approach first performs feature-to-transaction modality fusion via **virtual feature tokens**, followed by transaction-to-temporal modality fusion via **virtual transaction tokens**. The details of this mechanism are presented in Sec. 3.3.2.

Transaction Embedding Formulations. The architectural differences among the 1D, 2D, and 3D model variants lead to distinct computational formulations of the transaction-level embedding, summarized in Table 1 and detailed in Sections 3.2 and 3.3.

3.2 Three Transformers

3.2.1 Temporal Transformer

As depicted in Fig. 2 (a), we feed a transaction sequence $\mathcal{S}$ into the Temporal Transformer $\text{TF}(\mathcal{S})$ after we obtain each transaction’s embedding $E^{\mathbf{tr}} \in \mathbb{R}^{1 \times d_{\mathbf{tr}}}$, where $d_{\mathbf{tr}}$ is the transaction embedding dimension. We choose the *decoder-only* Transformer [Vaswani et al., 2017, Radford et al., 2019] as the backbone model. Specifically, each transaction’s embedding is regarded as a token and fed into

Table 2: The time-related fields in each transaction.

Time Field	Explanation
time_gap	# days/hours between the current transaction and the previous transaction
month	transaction month
year	# years since 2000
day_of_week	relative days in a week
day_of_month	relative days in the current month

stacked standard Transformer decoder blocks (abbr. TFDecoder) with multi-head self attention:

$$\text{TF}(\mathcal{S}) = \text{TFDecoder}(\{E^{\text{tr}_1}, \dots, E^{\text{tr}_n}\}, \text{tr} \in \mathcal{S}), \quad (2)$$

$$E^{\text{tr}_{n+1}} = \text{TF}(\mathcal{S})[\text{tr}_n, :]. \quad (3)$$

The decoder-only architecture enables TGPT to auto-regressively generate future transactions, where tr_1 to tr_n are the historical transactions encoded by $\text{TF}(\mathcal{S})$, tr_{n+1} is next transaction to be generated. The output of $\text{TF}(\mathcal{S})$ for last transaction tr_n is the embedding of next transaction $E^{\text{tr}_{n+1}} \in \mathbb{R}^{1 \times d_{\text{tr}}}$.

Local Attention. To improve the scalability of $\text{TF}(\mathcal{S})$ over long transaction sequences, we empirically find that the local attention mechanism [Beltagy et al., 2020] has a decent balance between input sequence length and downstream task performance. Specifically, the local attention mechanism restricts each transaction in $\mathcal{S}$ only attends to transactions in its context window with length equal to w , reducing the time complexity of causally masked attention from $O(|\mathcal{S}|^2 \times d_{\text{tr}})$ to $O(w^2 \times d_{\text{tr}})$, where $w \ll |\mathcal{S}|$. Please refer to Sec. 5.4 for its performance comparison with full attention.

Note that we also explored the *encoder-only* architecture for $\text{TF}(\mathcal{S})$, and its next one transaction generation performance is similar to TFDecoder. Extended discussions on Transformer design choices are presented in Sec. 6.

3.2.2 Feature Transformer

According to our discussion in Sec. 3.1, the feature encoder is a separate tabular Transformer [Huang et al., 2020] similar to $\text{TF}(\text{tr})$ but dedicated to downstream-dependent feature encoding:

$$\text{TF}(\mathcal{F}) = \text{TFEncoder}(\text{Emb}(\mathcal{F})), \quad (4)$$

each $E_f \in \mathbb{R}^{1 \times d_f}$, $f \in \mathcal{F}$ is obtained by linear projection $\text{Emb}(\mathcal{F})$.

3.2.3 Metadata Transformer

The Metadata Transformer $\text{TF}(\text{tr})$ employs an encoder-only backbone with bi-directional attention as we need to fully exploit the interactions between different data fields [Huang et al., 2020]. Since the encoder architecture is standard, we emphasize more on the non-trivial part, which is encoding different types of metadata as uni-sized embeddings as Transformer input. According to Sec. 2.1, each transaction has two types of metadata $\mathcal{M}$ and $\mathcal{E}$. Therefore, we define $\text{TF}(\text{tr})$ as follows and present the details of two embedding layers subsequently:

$$\text{TF}(\text{tr}) = \text{TFEncoder}(\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E}), \mathcal{M}, \mathcal{E} \in \text{tr}). \quad (5)$$

The above formulation is used for TGPT-2D without downstream features and TGPT-3D-MTF (detailed in Sec. 3.3.1), as shown in Table 1.

Metadata Embedding A transaction’s metadata contains the basic information and key attributes commonly used in payment processing. To make the foundation model general, we only include key metadata in model training, i.e., transaction time, amount, processing codes. Though the location and merchant information are crucial metadata, we treat them as entities in $\mathcal{E}$ and encode them separately. Please refer to **Entity Embedding** for more details.

Among the remaining metadata, temporal information is particularly critical. The way time is encoded influences the model’s ability to capture shopping behavior within each transaction sequence

and the purchasing patterns over varying time spans. Guided by time encoding approaches for time series Zerveas et al. [2021] and point process Zhang et al. [2020] and observed temporal patterns in consumer payments, we propose to convert each transaction’s raw timestamp into the numerical fields listed in Table 2. Among these fields, `time_gap` quantifies the interval between consecutive transactions, directly reflecting shopping cadence. To capture seasonality and weekday/weekend effects, which are crucial for shopping activities, we further introduce four new fields to amplify these signals. `month` and `year` encode the absolute calendar position, while `day_of_week` and `day_of_month` capture relative date information. The unimodal and diverse time fields facilitate the shopping behavior modeling and simplify the time embedding together with other metadata.

After transforming timestamps into numerical values, all the metadata data types are either numerical or categorical. We thereby design the metadata encoder as follows:

$$\text{Emb}(\mathcal{M}) = \left\|_{c=0}^C \left(\mathbf{E}_{m_c} \right) \oplus \left\|_{n=0}^N \left(\text{MLP}(\log(m_n + \epsilon)) \right); m_c, m_n \in \mathcal{M}. \quad (6)$$

Specifically, we use a trainable embedding layer $\mathbf{E}_{m_c} \in \mathbb{R}^{|m_c| \times d_{\mathcal{M}}}$ to encode each categorical field $m_c \in \mathcal{M}$, where $|m_c|$ is the cardinality of m_c and $d_{\mathcal{M}}$ is the embedding dimension for every metadata field in `tr`. For each numerical field $m_n \in \mathcal{M}$, we use an MLP to project it into $\mathbf{e}_{m_n} \in \mathbb{R}^{d_{\mathcal{M}}}$. A log transformation is applied to $m_n + \epsilon$ to reduce data skewness, where ϵ is a small constant. All encoded metadata are stacked ($\|$ and $\oplus$) as a 2-D tensor with size $(C + N) \times d_{\mathcal{M}}$, where C and N are the number of categorical and numerical metadata fields, respectively.

Entity Embedding In a large-scale payment system, transactions can be grouped into segments based on their associated entities, as transactions tied to the same entity often exhibit pronounced similarities. For example, transactions originating from a common geographic region tend to reflect local economic activity [Aladangady et al., 2019]. Moreover, each merchant displays a distinctive transaction pattern and customer base. Thus, a well-optimized and informative entity representation will further enhance the transaction representation learning. We use a trainable embedding layer $\mathbf{E}_e$ to encode each entity $e \in \mathcal{E}$ into $d_{\mathcal{E}}$ -dimensional embedding where $d_{\mathcal{E}} = d_{\mathcal{M}}$:

$$\text{Emb}(\mathcal{E}) = \{\mathbf{E}_e\}, e \in \mathcal{E}. \quad (7)$$

The primary challenge in jointly learning entity embedding with TGPT is handling the high-cardinality entities (e.g., merchant), which leads to a massive embedding table. This significantly increases the model parameter size, thereby hindering its scalability and optimization. Taking the merchant ID embedding table as an example, 10M merchants with embedding size 128 has 1.3B trainable parameters. It will also yield a fat classification head when predicting future merchants, making the model optimization less efficient.

Motivated by the observation that merchants belonging to the same category or segments share similar properties and recent work in embedding table compression [Shi et al., 2020], we employ compositional embedding to reduce the parameters of the merchant embedding layer.

Compositional Embedding. For n unique values of the entity $e \in \mathcal{E}$, we use the hashing trick to map each entity id to k hashed values belonging to $(1, m)$, $m \ll n$. Then we initiate the embedding table $\mathbf{E}_e \in \mathbb{R}^{m \times d_{\mathcal{E}}}$, reducing the embedding table memory from $O(n \times d_{\mathcal{E}})$ to $O(m \times d_{\mathcal{E}})$. Please check Appendix A for the complete implementation details.

LLM-based Embedding Initialization. Instead of learning entity embeddings from scratch, $\mathbf{E}_e$ can also be initialized by embeddings pre-computed by another embedding model. In this paper, we explore initializing MCC embeddings using LLMs and MCC descriptions [Fan et al., 2025]. Please refer to Appendix C for technical details and Sec. 5.1 for its contribution on the downstream task.

Metadata Transformer Extended. The Metadata Transformer can be extended to encode task-specific features, when available, alongside metadata fields and entities. In this case, Eq. 5 is modified as follows:

$$\text{TF}(\text{tr}) = \text{TFEncoder}(\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E}) \oplus \text{Emb}(\mathcal{F}), \mathcal{M}, \mathcal{E}, \mathcal{F} \in \text{tr}). \quad (8)$$

This formulation is applied for TGPT-2D with downstream task features, as shown in Table 1.

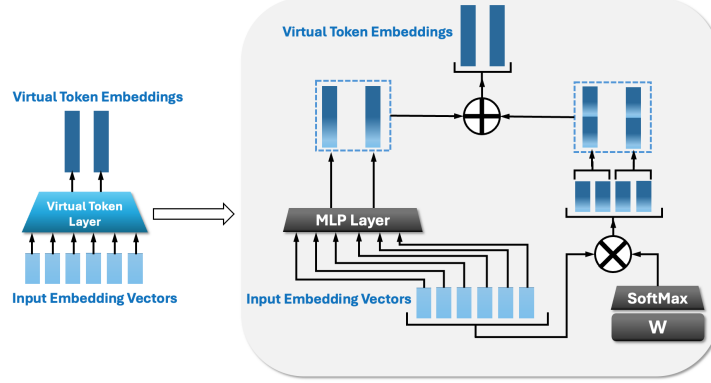


Figure 3: Virtual Token Layer (VTL).

3.3 Modality Fusion

As discussed in Sec. 3.1, integrating metadata and feature embeddings for each transaction presents two main challenges. First, metadata embeddings are few but high-dimensional, while feature embeddings are numerous but low-dimensional, making direct integration difficult. Second, after fusing these two modalities, the resulting transaction-level embedding needs to be sufficiently large to capture all relevant information; however, this significantly increases the computational cost of the temporal Transformer responsible for modeling inter-transaction dependencies. We devise the following two approaches to handle the above challenges.

3.3.1 Late Modality Fusion by Omitting Historical Feature Information

This approach omits historical feature information for generative tasks and inputs only the concatenated metadata embeddings, processed by $\text{TF}(\text{tr})$ and of controllable size, into $\text{TF}(\mathcal{S})$. For downstream transaction classification, where feature information is essential, we fuse the future transaction embedding $E^{\text{tr}_{n+1}} \in \mathbb{R}^{1 \times d_{\text{tr}}}$ (output by $\text{TF}(\mathcal{S})$) with the corresponding feature embeddings and pass them into $\text{TF}(\mathcal{F})$.

Since the transaction embedding size d_{tr} is usually much larger than the feature embedding size d_f , we can rescale $E^{\text{tr}_{n+1}}$ using an MLP to match d_f . However, this approach constrains the information bandwidth, leading to information loss and suboptimal performance, as shown in Table 4. Instead, we divide $E^{\text{tr}_{n+1}}$ evenly into multiple segments, each segment’s size matches d_f . These segments, together with all feature embeddings, are then input into the feature Transformer $\text{TF}(\mathcal{F})$. The output embeddings from $\text{TF}(\mathcal{F})$ are concatenated and connected to the prediction heads for downstream tasks. Because this solution sequentially employs a metadata Transformer, a temporal Transformer, and a feature Transformer, we refer to it as TGPT-3D-MTF. The architecture is illustrated in Fig. 2(d). For transactions 1 to n , where features are not attached, the transaction embedding formulation follows Eq. 5, as summarized in Table 1.

3.3.2 Early Modality Fusion via Virtual Tokens

TGPT-3D-MTF addresses the aforementioned two challenges by omitting historical feature information. However, this raises the question: is it possible to balance embedding dimension compatibility, information bandwidth, and computational feasibility while still retaining historical features, which may be valuable for both generative and downstream tasks? To achieve this, we introduce the **Virtual Token Layer (VTL)**, which transforms a set of embeddings into an arbitrary number of “virtual tokens”. These virtual tokens are soft representations that preserve essential information by providing sufficient bandwidth and flexible embedding sizes, and can be seamlessly passed to subsequent Transformer layers. As illustrated in Fig. 3, a VTL consists of both linear and nonlinear channels:

Linear. A softmax-based linear combination enables the selection and aggregation of the most informative input embeddings. Specifically, a parameter matrix $\mathbf{W}$ is processed through a softmax layer to produce a coefficient matrix, which is then multiplied with the embedding matrix to gener-

ate n new embeddings—each is a distinct linear combination of the originals. This linear operation preserves maximum information from the original embeddings and supports effective gradient propagation. Since it does not alter the embedding dimensionality, larger virtual tokens can be formed by stacking multiple embeddings, while smaller tokens can be created by segmenting an embedding as needed.

Nonlinear. A single-layer MLP with a nonlinear activation function is used to rescale the original embedding matrix into any desired number and size of output vectors.

The final virtual tokens are obtained via the summation of the linear and nonlinear components' outputs. In essence, the VTL functions as a scalable version of a ResNet [He et al., 2016] block: it enables arbitrary changes in token size, employs a linear path to retain original information, and uses a nonlinear path to enhance expressive power. We utilize VTL in two ways within our architecture:

Feature Tokenization. After the feature Transformer $\text{TF}(\mathcal{F})$ processes the feature embeddings, a VTL is utilized to generate v_f *virtual feature tokens* ($v_f \ll |\mathcal{F}|$) with higher dimensionalities:

$$\mathbf{E}^{\mathcal{F}} = \text{VTL}(\text{TF}(\mathcal{F})) = \text{VTL}(\text{TFEncoder}(\mathcal{F})) = [(V_1^{\mathcal{F}})^\top, (V_2^{\mathcal{F}})^\top, \dots, (V_{v_f}^{\mathcal{F}})^\top]^\top, \quad (9)$$

where tensor $\mathbf{E}^{\mathcal{F}} \in \mathbb{R}^{v_f \times d_{\mathcal{M}}}$ and vector $V_i^{\mathcal{F}} \in \mathbb{R}^{1 \times d_{\mathcal{M}}}$ for $1 \leq i \leq v_f$. $\mathbf{E}^{\mathcal{F}}$ is then combined with the $\text{Emb}(\mathcal{M})$ and jointly input to $\text{TF}(\text{tr})$.

Transaction Tokenization. The virtual feature embeddings $\mathbf{E}^{\mathcal{F}}$ are stacked with metadata embeddings $\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E})$, fed into a Metadata Transformer encoder, and then passed through a second VTL to produce *virtual transaction tokens*, which encapsulate all essential information for a single transaction. Thus, transaction representation $\mathbf{E}^{\text{tr}}$ becomes a set of virtual token embeddings:

$$\begin{aligned} \mathbf{E}^{\text{tr}} &= \text{VTL}(\text{TFEncoder}(\text{Emb}(\mathcal{M}) \oplus \text{Emb}(\mathcal{E}) \oplus \text{VTL}(\text{TFEncoder}(\mathcal{F})))) \\ &= [(V_1^{\text{tr}})^\top, (V_2^{\text{tr}})^\top, \dots, (V_{v_t}^{\text{tr}})^\top]^\top, \end{aligned} \quad (10)$$

where tensor $\mathbf{E}^{\text{tr}} \in \mathbb{R}^{v_t \times \frac{d_{\text{tr}}}{v_t}}$ and vector $V_i^{\text{tr}} \in \mathbb{R}^{1 \times \frac{d_{\text{tr}}}{v_t}}$ for $1 \leq i \leq v_t$. The v_t virtual transaction tokens of all n transactions are lined up sequentially, forming the $v_t n$ input vectors for the temporal Transformer $\text{TF}(\mathcal{S})$. Consequently, with this dual application of VTL, the architecture sequentially deploys a feature Transformer, a metadata Transformer, and a temporal Transformer; we refer to it as TGPT-3D-FMT, as illustrated in Fig. 2(e).

Key advantages of this design include: (1) Alignment of data modalities: virtual feature tokens can be sized to match $d_{\mathcal{M}}$, enabling effective fusion and joint processing by subsequent Transformers. (2) Flexible bandwidth and efficiency: v_f and v_t can be tuned to ensure sufficient information bandwidth while keeping computational costs manageable. (3) Optimized representation for sequential modeling: by representing each tr as a set of consecutive virtual tokens, the model avoids the prohibitive parameter and computational costs of large embedding sizes in $\text{TF}(\mathcal{S})$. Instead, it leverages Transformer architectures' strengths in handling long sequences, efficiently encoding both complex intra- and inter-transaction dependencies.

Further discussion on our design of the virtual token mechanism and its connections to visual tokenization and dual-channel feature crossing methodologies are presented in Sec 7.3. The formulation differences introduced by the virtual token mechanism for transaction-level embedding computation, compared with other TGPT variants, are summarized in Table 1.

3.4 Time Complexity Analysis

The time complexity of the TGPT model is primarily driven by the self-attention operation performed across different modalities. To simplify our analysis, we consider a single-head, single-layer self-attention architecture under context window w with $\mathcal{F}$ available. The complexities of the different TGPT variants are as follows:

- **TGPT-1D:** This model exclusively uses the $\text{TF}(\mathcal{S})$ module, as shown in Fig. 2. Its time complexity is: $O_{1D} = O(wd_{\text{tr}}^2 + w^2d_{\text{tr}})$. This consists of the projection layer complexity ($O(wd_{\text{tr}}^2)$) and the attention operation complexity ($O(w^2d_{\text{tr}})$).
- **TGPT-2D:** This model adds a $\text{TF}(\text{tr})$ to encode both merchant ($\mathcal{M}$) and feature ($\mathcal{F}$) modalities. The resulting complexity is: $O_{2D} = O(((|\mathcal{M}| + |\mathcal{F}|)d_{\mathcal{M}}^2 + (|\mathcal{M}| + |\mathcal{F}|)^2d_{\mathcal{M}})w) + O_{1D}$.

- **TGPT-3D-MTF:** This model only encodes $\mathbf{tr}_{n+1}$'s features. Its complexity includes the time complexities of $\text{TF}(\mathbf{tr})$ for $\mathcal{M}$, $\text{TF}(\mathcal{F})$ for $\mathbf{tr}_{n+1}$'s $\mathcal{F}$, and TGPT-1D: $O_{3\text{D-MTF}} = O((|\mathcal{M}|d_{\mathcal{M}}^2 + |\mathcal{M}|^2d_{\mathcal{M}})w) + O\left(\left(|\mathcal{F}| + \frac{d_{\mathbf{tr}}}{d_f}\right)d_f^2 + \left(|\mathcal{F}| + \frac{d_{\mathbf{tr}}}{d_f}\right)^2 d_f\right) + O_{1\text{D}}$. The term $\frac{d_{\mathbf{tr}}}{d_f}$ denotes the number of segments from the $E^{\mathbf{tr}_{n+1}}$ embedding fed into the $\text{TF}(\mathcal{F})$.
- **TGPT-3D-FMT:** This model applies the VTL for feature and transaction tokenization. Its computation includes $\text{TF}(\mathcal{F})$ for w $\mathbf{trs}$, $\text{TF}(\mathbf{tr})$ for $|\mathcal{M} + v_f|$ tokens of w $\mathbf{trs}$, and $\text{TF}(\mathcal{S})$ for $v_t w$ tokens with dimension $\frac{d_{\mathbf{tr}}}{v_t}$. The total time complexity is: $O_{3\text{D-FMT}} = O\left(\left(|\mathcal{F}|d_f^2 + |\mathcal{F}|^2d_f\right)w\right) + O\left(\left(|\mathcal{M}| + v_f\right)d_{\mathcal{M}}^2 + \left(|\mathcal{M}| + v_f\right)^2d_{\mathcal{M}}\right)w + O\left(v_t w\left(\frac{d_{\mathbf{tr}}}{v_t}\right)^2 + (v_t w)^2\frac{d_{\mathbf{tr}}}{v_t}\right)$.

Although TGPT-2D is more expensive than TGPT-1D due to the Transformer encoding of metadata and features, and TGPT-3D-FMT is less efficient than TGPT-3D-MTF because it encodes every $\mathbf{tr}$'s feature, we find that the added model capacity delivers substantial downstream gains (see Sec. 5.1). Notably, FMT incurs only $\sim \times 2.5$ higher inference latency than MTF while remaining within the company's service-level agreement (SLA), preserving production viability.

We emphasize that novel information fusion techniques in TGPT-3D-FMT makes it significantly more efficient than TGPT-2D, as we have a large amount of transaction features requiring relatively small embedding sizes, and TGPT-3D-FMT's transaction embedding sizes are much greater than its attention window size:

Theorem 1. *If $1 < d_f \ll d_{\mathcal{M}}$, $1 < v_f, v_t$, $|\mathcal{M}| \ll |\mathcal{F}|$, and $1 < w \ll \frac{d_{\mathbf{tr}}}{v_t}$, then $O_{3\text{D-FMT}} \ll O_{2\text{D}}$.*

The proof of Theorem 1 is in Appendix B.

3.5 Model Output and Loss Function

As shown in Fig. 2, for TGPT-1D and -2D, $E^{\mathbf{tr}_{n+1}}$ is connected to the prediction heads to generate metadata fields and downstream task labels. For TGPT-3D-MTF, the output of $\text{TF}(\mathcal{F})$ is only used for downstream task prediction, and the output of $\text{TF}(\mathcal{S})$ is connected to metadata prediction heads. For TGPT-3D-FMT, the last k output vectors of $\text{TF}(\mathcal{S})$, namely the Temporal Transformer encodings of the last k virtual tokens $V_{v_t-k}^{\mathbf{tr}}, V_{v_t-k+1}^{\mathbf{tr}}, \dots, V_{v_t}^{\mathbf{tr}}$ of the $(n+1)$ th transaction, are concatenated and connected to the prediction heads.

TGPT has two types of optimization objectives: the self-supervised objective for the next $\mathbf{tr}$ prediction (generative) and the supervised objective with downstream task labels (predictive). The self-supervised training follows the heuristics that $\mathbf{tr}_i$ is dependent on $[\mathbf{tr}_1, \dots, \mathbf{tr}_{i-1}]$ for $\mathbf{tr}_i \in \mathcal{S}$. Optimizing TGPT via letting it predict key properties of future transactions based on historical transactions enables the model to learn the invariants behind transacting dynamics. Meanwhile, we believe the joint optimization of self-supervised and supervised signals can mutually benefit both the generative and predictive tasks.

To make self-supervised learning generalize to all transactions and downstream tasks, we pick *four* essential fields from $\mathcal{M}$ to be generated for future transaction: *time_gap*, *amount*, *merchant*, and *MCC*. *time_gap* uses its original integer value (numerical); transaction amount is an integer representing US dollar cents (numerical); each merchant/MCC has a unique identifier (categorical). The prediction head and loss function for self-supervised training are designed according to the field type we would like to predict:

$$\mathcal{L}_{\text{sst}}(\text{num}) = (\text{MLP}(E^{\mathbf{tr}_{n+1}}) - \log(y_{\text{num}} + \epsilon))^2, \quad (11)$$

$$\mathcal{L}_{\text{sst}}(\text{cat}) = - \sum_{c=1}^C y_c \log(\text{softmax}(\text{MLP}(E^{\mathbf{tr}_{n+1}}))_c). \quad (12)$$

Weight Tying. Since the prediction head for high-cardinality categorical entity (e.g., merchant) introduces a massive amount of trainable parameters, we resort to weight tying technique Press and Wolf [2017] to reduce model size and improve training/inference efficiency. Specifically, instead of using a dedicated prediction head MLP, the entity logits is computed via $\mathbf{E}_e(E^{\mathbf{tr}_{n+1}})^{\top}$, where $\mathbf{E}_e$ is the trainable embedding layer parameters of entity e from $\text{Emb}(\mathcal{E})$.

In TGPT, the supervised signal comes from downstream tasks inside the company which is usually a classification problem. The final loss function is the composition of self-supervised loss and an optional supervised loss based on the availability of downstream tasks, plus regularization losses: weight decay and the VTL matrix orthogonality losses (Sec 3.3.2):

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{sst}}(\text{num}) + \lambda_2 \mathcal{L}_{\text{sst}}(\text{cat}) + \lambda_3 \mathcal{L}_{\text{st}} + \lambda_4 \mathcal{L}_{\text{weight}} + \lambda_5 (\mathcal{L}_{\text{orth}}^{\mathcal{F}} + \mathcal{L}_{\text{orth}}^{\text{tr}}), \quad (13)$$

where the λ s are the weight hyper-parameters; $\mathcal{L}_{\text{st}}$ is dependent on the availability of downstream task supervision signal; $\mathcal{L}_W$ stands for the weight decay loss. VTL matrix orthogonality losses for virtual feature tokens and virtual transaction tokens are given by

$$\mathcal{L}_{\text{orth}}^{\mathcal{F}} = \|\mathbf{E}_{\text{lin}}^{\mathcal{F}} (\mathbf{E}_{\text{lin}}^{\mathcal{F}})^{\top} - \mathbf{I}_{v_f}\|_{\text{F}}^2, \quad (14)$$

$$\mathcal{L}_{\text{orth}}^{\text{tr}} = \|\mathbf{E}_{\text{lin}}^{\text{tr}} (\mathbf{E}_{\text{lin}}^{\text{tr}})^{\top} - \mathbf{I}_{v_t}\|_{\text{F}}^2, \quad (15)$$

where $\|\cdot\|_{\text{F}}^2$ denotes Frobenius norm, and $\mathbf{E}_{\text{lin}}^{\mathcal{F}}$ and $\mathbf{E}_{\text{lin}}^{\text{tr}}$ respectively denote the linear combination part of the feature and transaction virtual tokens.

In Sec. 6, we have additional discussion on model design and lessons learned through model training.

4 Training Recipe

4.1 Data Preparation

We collect and process billion-scale transactions in strict compliance with data security and privacy regulations, ensuring no access or disclosure of personally identifiable information. We segment a full transaction sequence into length- l subsequences, padding those shorter than l and discarding those below l_{min} . To have a fair comparison with baselines, we curate three datasets for three evaluation tasks shown in Table 3, then train and evaluate TGPT on each of them separately. The test data consists of transaction sequences collected from a period subsequent to the training data.

T-JGC: it is our primary evaluation setting based on a key revenue-driven product. It is used to demonstrate TGPT’s transaction generation and anomaly detection performance against a strong production model. T-JGC data merges mainstream (metadata only) with downstream transactions (metadata + features). The merger yields a richer payment history but introduces rows that contain metadata only (Fig. 1), increasing modeling complexity.

T-RES: it evaluates TGPT’s dining trajectory generation performance and merchant embedding quality using side information. T-RES data uses offline, card-present restaurant transactions only with metadata. We filter out sequences where the same merchant appears in $> 80\%$ of transactions to filter naïve recurring patterns and prevent model overfitting.

T-MCC: to have a fair comparison of TGPT and LLM-based foundation model, we benchmark them under the future transaction MCC prediction task. T-MCC data is curated by sampling 1M transaction sequences with diverse types of MCCs (~ 800 classes), which is appropriate for LLM fine-tuning.

4.2 Model Config and Training Stats

Due to the large design space of TGPT and its dependence on proprietary data, infrastructure capacity, and downstream tasks, there is no single silver-bullet model configuration. Instead, we report the search space of key model components explored during tuning in Table 8, along with model sizes of the most performant configurations trained on the T-JGC dataset: with the embedding layer and prediction heads excluded, TGPT-1D and TGPT-2D contain 7.5M parameters, while TGPT-3D-FMT contains 10.3M parameters.

We use the AdamW optimizer, CosineAnnealingLR scheduler, and gradient clipping during training; weight decay and dropout are enabled with fixed default values. The batch size is empirically determined based on VRAM capacity and training stability. An extended discussion of the design choices we made and the lessons learned during TGPT training can be found in Sec. 6.

All models are trained using PyTorch 2.1 with DistributedDataParallel on GPU clusters equipped with NVIDIA A100 80GB GPUs and CUDA 12.3. The most performant TGPT-1D, 2D, and 3D configurations require 362, 712, and 480 GPU-hours, respectively, to converge on the T-JGC dataset.

Table 3: Dataset statistics.

Dataset	Seq. len	# Train Seqs	# Test Seqs	# Merchants	# Classes	$ \mathcal{F} $
T-JGC	8	100M	10M	3M	2	~ 400
T-RES	16	200M	20M	500K	500K	-
T-MCC	16	1M	100K	50K	~ 800	-

5 Model Evaluation

We evaluate TGPT’s foundational capability through three diverse tasks towards different payment use cases: **T-JGC**: future transaction generation jointly with downstream anomaly transaction detection (Sec. 5.1); **T-RES**: dining transaction trajectory generation and merchant representation learning (Sec. 5.2); **T-MCC**: benchmarking MCC prediction performance with fine-tuned LLMs (Sec. 5.3). Sec. 5.4 shows the empirical impact of several scalability designs in TGPT. Our findings and insights through experiments are highlighted in *italics*.

5.1 Transaction Generation and Anomaly Detection

Evaluation Setting. With the downstream feature available, we assess TGPT’s capacity to learn comprehensive payment dynamics through generating future transaction metadata and its anomaly probability (Table 4). For MCC and merchant, the average Recall@1 is reported. It is particularly challenging for merchant prediction, as TGPT must identify the correct one from 3M distinct options. MAE and MSE are reported for time and amount prediction. A proprietary metric (**AD**) is used to assess **Anomaly Detection** performance, which measures the accuracy under an operational threshold. We omit the detailed formulation of **AD** and only report the relative improvement (**RI**) of each model w.r.t. the production model for company policy compliance. Importantly, the production model is trained on the downstream dataset without missing fields, while TGPT is trained on the merged dataset containing bubbles.

We compare TGPT variants along with two representative baselines in this experiment. For transaction generation, a Transformer-based sequential recommendation model, **Transformers4Rec** Moreira et al. [2021], is trained to generate future MCCs. To investigate the contribution of sequential information to anomaly detection performance, we train an instance-level (per transaction) **Feat-Transformer** that only processes a single transaction without temporal context. To evaluate the contribution of individual architectural components of TGPT, we compare the performances of the following model variants:

- **TGPT-1D**: with/without downstream features (Fig. 2(b)).
- **TGPT-2D**: with/without downstream features (Fig. 2(c)).
- **TGPT-3D-MTF** (Fig. 2(d)): with segments or MLP-Scaling.
- **TGPT-3D-FMT** (Fig. 2(e)):
 - **FMVTL**: with Feature and Metadata VTLs; the last $k = 1$ virtual temporal tokens are connected to prediction heads (unless otherwise specified, $k = 1$ is by default).
 - **FMVTL-2**: same with **FMVTL** except $k = 2$.
 - **FMVTL-nonlin**: Feature and Metadata VTLs only adopts the nonlinear (MLP) component.
 - **FMVTL-lin-map**: Feature and Metadata VTLs are replaced by simple linear mappings.
 - **FVTL**: with Feature VTL but without Metadata VTL.
 - **FMVTL+LLM**: The MCC embeddings are initialized using LLM-generated representations, where ISO MCC descriptions serve as input prompts.

Transaction Generation. Table 4 shows that TGPT-1D/2D w/o Feat achieves reasonable performance on metadata prediction, correctly identifying MCCs (out of ~ 800 categories) with near 50% accuracy and merchants (out of 3M) with $> 30\%$ accuracy. Transformers4Rec yields notably lower accuracy than TGPT-1D/2D for future MCC generation. As the merchant ID possesses much higher cardinality than MCC and is therefore more difficult to model, merchant prediction result for Transformers4Rec is omitted. Across amount and time prediction tasks, the 2D variants consistently achieve lower prediction errors than TGPT-1D, suggesting that the additional Metadata Transformer

Table 4: Performance comparison of TGPT and its variants on transaction generation and anomaly detection (AD) tasks. For compliance, only the relative improvement (RI) over the production model performance is shown for the anomaly detection task.

Model	AD (%)	MCC (%)	Mrch (%)	Amount		Time	
	RI $\uparrow$	Rec@1 $\uparrow$	Rec@1 $\uparrow$	MAE $\downarrow$	MSE $\downarrow$	MAE $\downarrow$	MSE $\downarrow$
Transformers4Rec	-	33.46	-	-	-	-	-
Feat-Transformer	+7.7	-	-	-	-	-	-
TGPT-1D							
- w/o Feat	-90.5	48.86	31.60	1.29	3.84	0.1350	0.0355
- w/ Feat	-9.1	49.25	30.46	1.30	3.93	0.1370	0.0282
TGPT-2D							
- w/o Feat	-87.0	48.89	31.41	1.27	3.78	0.0702	0.0100
- w/ Feat	+7.3	49.17	30.38	1.26	3.79	0.0740	0.0095
TGPT-3D-MTF							
- Segments	+14.6	48.23	32.41	1.33	4.05	0.1420	0.0329
- MLP-Scaling	+9.5	48.17	32.29	1.32	4.05	0.1400	0.0325
TGPT-3D-FMT							
- FMVTL	+19.2	49.94	32.08	1.23	3.71	0.0893	0.0120
- FMVTL-2	+15.5	50.12	<u>32.70</u>	<u>1.24</u>	<u>3.73</u>	0.0800	0.0098
- FMVTL-nonlin	+6.7	49.55	31.20	1.30	3.86	0.0901	0.0141
- FMVTL-lin-map	+0.2	49.62	31.06	1.26	3.78	0.0880	0.0152
- FVTL	-12.9	48.45	30.91	1.28	4.16	0.0767	0.0111
- FMVTL + LLM	+22.5	<u>50.01</u>	32.73	1.25	3.77	0.0686	0.0072

enables the modeling of more fine-grained information, while delivering comparable performance on MCC and merchant prediction. We further observe that feature-free models perform on par with their feature-augmented counterparts across most metrics, suggesting that auxiliary feature information is not fully exploited in generative tasks. TGPT-3D-MTF variants show no obvious advantage over TGPT-1D/2D w/ Feat in most cases, indicating that the MTF approach to feature fusion is suboptimal for generative tasks. TGPT-3D-FMT achieves the best performance across all metrics, underscoring the critical role of feature integration and a well-designed modality fusion architecture.

Remarks. 1. Domain-engineered features provide substantial enhancement to the generative task. 2. Effective multimodal integration is non-trivial: features maximize prediction performance the most when they are properly fused with metadata before temporal modeling. 3. FMT with dual VTLs successfully leverages the predictive signals of domain features when they are appropriately integrated.

Anomaly Detection. Domain features play a more critical role in this task: TGPT-1D w/ Feat exhibits degraded performance on metric AD compared to both the production and Feat-Transformer baselines. In contrast, the 2D variants and Feat-Transformer explicitly use transformers to model interactions among features, enabling them to outperform the 1D architecture. Surprisingly, TGPT-2D w/ Feat shows no gains over the Feat-Transformer, despite utilizing full sequences versus single transactions, highlighting the challenge of effectively utilizing temporal information. Sec. 6 presents further analysis from an information bandwidth perspective. The MTF fusion strategy in TGPT-3D-MTF provides considerable gains, achieving substantial relative improvement over the baselines, proving successful at integrating sequential metadata with features. However, this architecture is constrained by its inability to incorporate feature information from historical transactions. Finally, TGPT-3D-FMT emerges as the optimal configuration, delivering the most significant performance improvements on the anomaly detection metric while simultaneously maintaining superior generative performances.

Remarks. 1. Naive feature concatenation for each transaction (2D) fails to leverage sequential and domain feature information for the classification objective. 2. Sophisticated modality fusion architectures (3D-MTF/FMT) are more critical for downstream classification than for the generative task. 3. MTF/FMT is unaffected by data bubbles. 4. The dual VTL structure in FMT enables more effective multi-modal representation learning, successfully bridging the gap between generation and classification objectives.

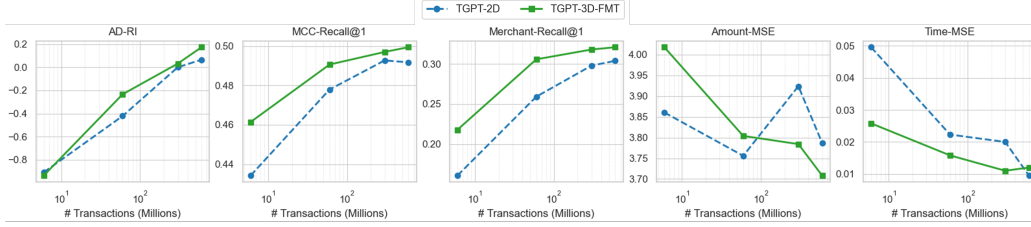


Figure 4: Data scaling behavior: performance metrics as a function of the number of training transactions. The x-axis is in log scale.

Table 5: (a) Future restaurant and (b) future location prediction accuracy of TGPT-2D given previous dining trajectories.

(a)	SASRec	TGPT-1D	TGPT-2D	TGPT-2D -m-only	(b)	Zip code	City	State
Rec@1	11.9%	12.8%	14.2%	13.7%	Top-1	28%	42%	84%
Rec@50	38.5%	42.5%	45.6%	43.8%	Top-10	60%	69%	91%

Ablation Study. We systematically evaluate the effectiveness of key architectural components for modality fusion. TGPT-3D-MTF: Segmented transaction embeddings maintain complete information bandwidth between the transaction embedding and the feature Transformer, outperforming MLP-scaling on metric **AD**, confirming that information compression degrades downstream performance. TGPT-3D-FMT: FMVTL-nlin eliminates the linear combination in VTL, converting it to a standard MLP. Universal performance drops indicate that the linear pathway is essential for gradient flow and information preservation; FMVTL-lin-map replaces the full VTL with basic linear projection for dimension matching. Further degradation on **AD** demonstrates that nonlinear expressiveness is crucial, validating the linear+nonlinear design. FVTL removes metadata VTL entirely, causing the most severe performance drops across most classification and generation metrics. This confirms that both FM and FMT fusion stages are necessary for effective multi-modal integration.

FMVTL+LLM yields modest improvements on generative metrics and delivers notable gains over FMVTL trained from scratch on the anomaly detection task, demonstrating that LLM-derived semantic information enhances downstream performance by leveraging external knowledge to better understand transaction dynamics.

LLM-initialized MCC Embedding. FMVTL+LLM yields modest improvements on generative metrics and delivers notable gains over FMVTL trained from scratch on the classification task, demonstrating that LLM-derived semantic information enhances downstream performance by leveraging pre-existing knowledge to better understand transaction dynamics.

Data Scaling Law. We investigate model performance under increasing training transaction volumes by training multiple instances of TGPT-2D/3D. As shown in Fig. 4, TGPT’s performance improves consistently with larger training datasets across all tasks, indicating that the data scaling law is one key driver of further improving the generative and predictive power of transaction foundation models. Notably, the downstream task metric **AD** exhibits the greatest performance gains under the 3D configuration. On the smallest dataset, however, the 3D model performs comparably to the 2D model, indicating that the advantages of the 3D architecture emerge more prominently with larger data volumes. Given the modest size of our current models, there remains substantial room for scaling model capacity, which we leave for future work.

5.2 Dining Trajectory Generation

We evaluate the dining trajectory generation capability of TGPT via training TGPT on a restaurant transaction only data (T-RES) and letting TGPT predict the next restaurant. Since there is no personal identifiable information in the input data and our data and model weights are secured internally, the dining trajectory generation task only reflects the foundation transaction modeling capability of TGPT from a collective perspective without compromising individual privacy.

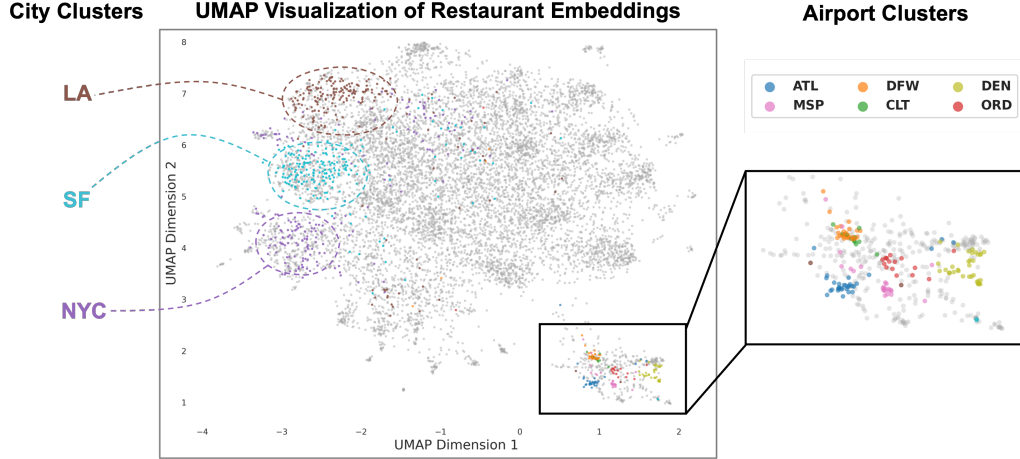


Figure 5: UMAP visualization of the TGPT merchant embeddings of the top 10K restaurants ranked by transaction volume, plus highlighted restaurants from major US cities (left) and airports (right).

Quantitative Evaluation. We select SASRec [Kang and McAuley, 2018], a classic sequential recommendation model trained on merchant IDs, as the restaurant prediction baseline. Since there is no downstream features in this data, we only evaluate the TGPT-1D and -2D models.

Table 5(a) shows the restaurant prediction performance under Recall@1 and Recall@50. The results show that all TGPT model variants outperform SASRec. Notably, among 45.6% test sequences, TGPT-2D can rank the exact future restaurant as the top 50 candidates from 500K restaurants. Meanwhile, TGPT-2D outperforms TGPT-1D, demonstrating metadata Transformer encoder is better than naïve MLP-based embedding layer. TGPT-2D optimized with all prediction heads is better than TGPT-2D only optimized with the merchant prediction head (TGPT-2D-m-only). *The results collectively verify that the MMTT foundation model’s multi-field optimization is advantageous and necessary: predicting auxiliary metadata fields improves merchant prediction.*

Thanks to the additional restaurant location information we have, we also evaluate the future dining location prediction accuracy of TGPT-2D using restaurant prediction as a proxy. Specifically, there is no location info in TGPT input/output, and the accuracy is computed by checking if the location of generated top-k restaurants match the location of the ground truth restaurants. The evaluation results in Table 5(b) are very promising: we can successfully predict the future dining locations with Accuracy@10=69% even if TGPT does not ingest any location-related info. The results further suggest that *TGPT is able to encode consumer payment patterns and merchant similarities and thereby benefit dining trajectory generation.*

Qualitative Evaluation. The stellar quantitative results on dining trajectory generation suggest that TGPT learns the inherent similarity between restaurants. To validate it, we visualize the restaurant embeddings learned from TGPT via UMAP [McInnes et al., 2018] in Fig. 5. Based on the heuristic that the restaurants belonging to the same dining trajectory should have location proximity with each other, we highlight restaurants from selected US cities and airports to demonstrate the proximity from a local mobility and air mobility perspective, respectively.

From the local mobility perspective, we highlight 200 randomly sampled restaurants from each of three major US cities: Los Angeles (LA), San Francisco (SF), and New York City (NYC). In Fig. 5 (left), most restaurants belonging to the same city are closely clustered together, and three city-based clusters have explicit boundaries between each other. It proves that TGPT embeddings learned by predicting future transactions encode the location proximity of different restaurants very well.

From the air mobility perspective (Fig. 5 (right)), we highlight restaurants from the top 2 hub airports ranked by passenger numbers from 3 major US airlines (Delta: ATL and MSP; American: DFW and CLT; United: DEN and ORD). The UMAP plot shows that airport-based restaurants are densely clustered and clearly separated with other restaurants, suggesting the airport-based restaurants have their unique properties and are different from most day-to-day dining places.

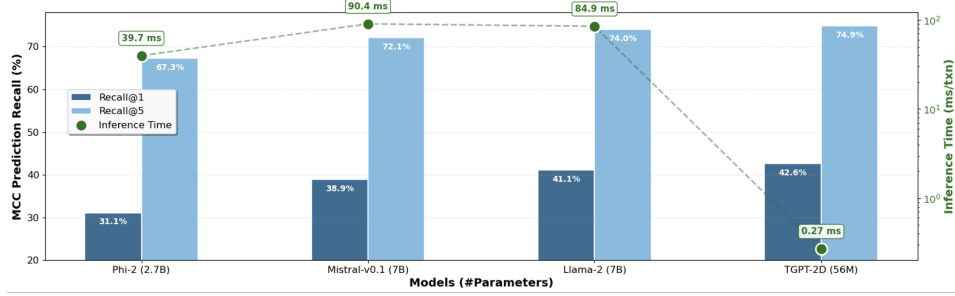


Figure 6: The field prediction task performance and inference speed comparison between TGPT and other models.

We have the following three observations from the individual airport view: (1) Restaurants located within the same airport tend to form distinct clusters, and for hub airports operated by the same airline, these clusters appear relatively close to each other. (2) The embeddings of DFW (orange) and CLT (green) restaurants show substantial overlap, which could be due to the high passenger traffic and shared hub status of these two largest American Airlines hubs. (3) ORD (Chicago) restaurant embeddings are positioned between those of DEN, DFW, and MSP. This intermediate positioning may reflect ORD’s role as a major hub for both United and American Airlines, as well as potential similarities in passenger demographics with nearby airports such as MSP.

To summarize, the UMAP visualization explains restaurant embeddings learned by TGPT do encode the proximity of restaurants along the dining trajectories.

5.3 Merchant Category Prediction

Besides the dining trajectory generation (i.e., merchant prediction) task, we evaluate the transaction generation performance on another representative transaction field: merchant category code (MCC). The curated T-MCC data includes transactions covering ~ 800 MCCs. Instead of comparing classic sequential prediction baselines, we implement LLM-based baselines, as LLM-based foundation models are actively explored by many industry practitioners.

Since this task does not have downstream features, we compare TGPT-2D’s next MCC prediction performance with representative open weights LLMs [Touvron et al., 2023, Jiang et al., 2023, Javaheripi et al., 2023]. For LLM baselines, we describe each historical transaction’s metadata using plain English language following a table-to-text template [Hegselmann et al., 2023] and use it as the LLM input. LLM is fine-tuned to predict the next transaction’s MCC, where each MCC is registered as a new special token in each LLM’s tokenizer. Please refer to Appendix D for more LLM baseline implementation details.

In Fig. 6, we report the MCC prediction performance (blue bars) of different models and their respective inference time (green dots) on a single NVIDIA A100 GPU (80GB) instance. For a fair comparison, we incorporate the embedding layer size when computing TGPT-2D’s model size. TGPT-2D obtains better prediction performance than the best LLM (Llama2-7B) with 92% fewer parameters (56M vs. 7B) and 300 $\times$ faster inference speed (0.27 ms vs. 84.9 ms). It demonstrates that *foundational models tailored to MMTT data is cost-efficient and practical comparing to LLM-based solutions on the MCC prediction task*. Meanwhile, we are impressed by the MCC prediction performance of fine-tuned LLMs and are optimistic about LLM-based foundation models on suitable transaction downstream tasks.

5.4 Scalability Evaluation

In Sec. 3, we devise and employ various techniques to improve TGPT’s scalability. Table 6 shows their relative impact on peak VRAM usage and anomaly detection performance w.r.t. the TGPT-3D-FMT base model without any scalability design. Since the uncompressed high-cardinality merchant embedding table takes numerous parameters, the compositional embedding and weight tying substantially reduce TGPT model size and thereby peak memory usage. Meanwhile, we observe that tying the softmax weight with the embedding table only has a minor negative influence on down-

Table 6: The relative single-GPU peak VRAM usage and anomaly detection performance of different scalability enhancing techniques w.r.t the base TGPT-3D model.

Scalability Design	Peak Memory Usage	Downstream Performance
TGPT-3D-base	-	-
+ compositional embedding (Sec. 3.2.3)	-22.6%	+1.9%
+ local attention (Sec. 3.2.1)	-24.9%	+2.8%
+ weight tying (Sec. 3.5)	-73.8%	-0.6%

stream performance (-0.6%), proving that weight tying is a practical solution. More surprisingly, the compositional embedding and local attention reduce the embedding parameters and context window but yield slightly better performance on the downstream task. We speculate that *compact model alleviates the overfitting, and the selected anomaly detection task may not favor long-term transaction dependencies*.

6 Discussion on Model Design and Training

We have been carrying out numerous experiments with different TGPT model variants and training approaches. Due to privacy and legal compliance, we cannot disclose full details of the model architecture and source code. We distill and share the following practical guidelines aimed at improving training stability, task performance, and computational efficiency. To improve readability, we present the material following the model’s forward propagation—i.e., in the order that data flows through each component.

Normalizations. Applying normalization when connecting embeddings to transformer modules is essential for both training stability and performance. For TGPT, BatchNorm consistently outperformed LayerNorm, likely due to the highly diverse cardinalities across fields and the multi-modal nature of the data. Within each transformer, we adopt a Pre-LN architecture as recommended by [Xiong et al., 2020].

Positional Encoding for Non-sequential Fields. For $\text{TF}(\text{tr})$ and $\text{TF}(\mathcal{F})$, where input fields lack intrinsic order, we compared one-hot and sinusoidal positional encodings. One-hot encodings delivered slightly better performance.

Information Bandwidth. In the **TGPT-2D** configuration, the output embedding size d_{tr} of the *Metadata Transformer* $\text{TF}(\text{tr})$ determines its communication bandwidth with the subsequent *Temporal Transformer* $\text{TF}(\mathcal{S})$, as denoted by $\text{TGPT-2D-}d_{\text{tr}}$ in Fig. 4. Increasing d_{tr} from 64 to 128 yields substantial performance improvements—particularly for metrics A and B—underscoring the importance of high-bandwidth information transfer. However, because the key-field embeddings produced by $\text{TF}(\text{tr})$ are concatenated to form the full transaction-level embedding that serves as input to $\text{TF}(\mathcal{S})$, enlarging d_{tr} proportionally increases the embedding dimensionality and, consequently, the computational cost of $\text{TF}(\mathcal{S})$. Scaling s_{tr} further from 128 to 256 results in OOM errors on an NVIDIA A100 GPU (80 GB).

Integrating Metadata Transformer Outputs. As detailed in **Information Bandwidth**, constructing a transaction-level embedding from the $\text{TF}(\text{tr})$ output for input into the $\text{TF}(\mathcal{S})$ poses a significant design challenge in TGPT-2D. We evaluate five integration strategies (design choices for the Integrate operation in Table 1) and observe their respective trade-offs:

- **Concatenation.** Directly concatenating all output embeddings from the $\text{TF}(\text{tr})$ to form the transaction-level embedding offers maximum information bandwidth, but results in prohibitively large embeddings and frequent out-of-memory (OOM) errors.
- **MLP compression / linear mapping.** All output embeddings are concatenated and then projected to an arbitrary dimensionality using an MLP block (with linear or nonlinear activation). This allows flexible bandwidth control, but the large input dimensionality produces dense parameter matrices prone to training instability and overfitting, leading to sub-optimal performance.
- **Pooling (summation/average/max).** Element-wise pooling is applied across all outputs of the $\text{TF}(\text{tr})$. This approach is computationally efficient and memory-friendly, but aggregates hundreds of heterogeneous field embeddings without filtering, introducing substantial noise. The resulting

transaction-level embedding has the same dimensionality as a single field embedding, severely limiting information bandwidth.

- **Field-embedding selection + pooling.** To reduce noise, only embeddings from a small set of essential fields are pooled. While this improves signal quality, the dimensionality remains constrained to that of a single field embedding, limiting bandwidth. Additionally, field selection risks information loss: it is difficult to identify the most critical fields among hundreds, and pooling inherently discards information.
- **Field-embedding selection + concatenation.** This approach replaces pooling with concatenation for the selected key-field embeddings, reducing noise and offering more flexible bandwidth. While pooling-related information loss is avoided, selection still risks discarding relevant features and requires strong heuristics, often informed by domain expertise.

These trade-offs motivate the design of **TGPT-3D-FMT**, which incorporates the *virtual token mechanism* to decouple information bandwidth from embedding size, thereby providing full controllability in information flow between transformers of different modalities. Among the above methods, the option of field-embedding selection + concatenation provided a relatively good balance of stability, efficiency, and accuracy when combined with carefully selected fields based on domain knowledge and extensive experimentation. Table 7 in Appendix E provides an empirical comparative summary of the strengths and limitations associated with the five integration strategies.

Design Choices for Temporal Transformer. We investigated the encoder-only Transformer architecture for $\text{TF}(\mathcal{S})$ with bi-directional attention. During training, the last j transactions of each sequence are masked for prediction, where $1 < j < |\mathcal{S}|$, as the BERT-style [Devlin et al., 2019]. For single-step prediction ($j = 1$), the encoder-only model’s performance on generating transaction attributes (merchant, MCC, time, amount) is comparable to a decoder-only model. To generate multiple future transactions, the encoder-only model operates recursively, predicting the next j transactions while discarding the oldest j to maintain a fixed sequence length $|\mathcal{S}|$.

Empirically, this recursive approach allows the encoder-only model to outperform the decoder-only model for short-term forecasting ($1 < j < |\mathcal{S}|$). However, its primary weakness is context erosion: after generating $|\mathcal{S}| - 1$ transactions, the original input is completely lost, causing subsequent predictions to lack grounding. In contrast, the decoder-only model’s auto-regressive nature retains the full historical input, making it superior for long-horizon forecasting ($j > |\mathcal{S}|$). Therefore, the optimal Transformer architecture depends on the prediction horizon required by the specific use case.

Anomaly Prediction Loss. Given that anomalies are rare events, the downstream classification labels are highly imbalanced. We tested focal loss [Lin et al., 2018] and class weighting strategies, but found no improvement over standard cross-entropy loss with equal class weights. We attribute this to the large absolute number of anomaly samples in our dataset, which mitigates the impact of class imbalance despite its rarity.

Parameter Precision and Quantization. We experimented with BF16 and FP32 precision during training. BF16 helps constrain overfitting on small datasets but limits performance growth when scaling data volume. In contrast, FP32 tends to overfit small datasets but better preserves scaling gains with larger data.

These lessons directly informed the architectural and optimization choices in TGPT-3D, improving stability, generalization, and scalability across diverse data regimes.

7 Related Work

7.1 Deep Learning for Transaction Data

Most of the transaction data has rich entity-connectivity and temporal information, which make it suitable for graph or sequential neural encoders. The graph-based methods regard the transactions between different entities as edges in a graph (e.g., account-to-account, account-to-merchant). Graph Neural network (GNN)-based [Liu et al., 2021, Li et al., 2023] approaches are used to learn transaction or other entity representations. The graph models capture contextual information in transaction networks and are ideal for tasks like item recommendation and anomaly detection with transaction data. Sequential methods encode a series of transactions along the temporal dimension and leverage RNN [Babaev et al., 2019, Skalski et al., 2023] or Transformer [Padhi et al., 2021, Zhang et al.,

2023, Brüel Gabrielsson et al., 2025, Li et al., 2025, Aminian et al., 2025, Li et al., 2025] to encode the transaction sequences. The sequential models are either trained in a supervised fashion [Babaev et al., 2019, Aminian et al., 2025] to directly accommodate downstream tasks or together with self-supervised objectives that reconstruct or generate certain transaction properties [Padhi et al., 2021, Bazarova et al., 2024, Brüel Gabrielsson et al., 2025, Li et al., 2025]. The common downstream tasks for sequential modeling of transaction data include classification and regression.

In this work, we deal with consumer payment sequences with diverse downstream tasks. Therefore, we adopt the sequential modeling paradigm and train the model with self-supervised and supervised objectives. We highlight the differences between our work and previous work with sequential transaction encoders [Babaev et al., 2019, Skalski et al., 2023, Padhi et al., 2021, Zhang et al., 2023, Brüel Gabrielsson et al., 2025, Aminian et al., 2025, Li et al., 2025] as follows: 1) TGPT is trained on billions of real-world consumer-to-merchant transaction data and evaluated on different downstream tasks against production models; 2) instead of using discrete tokens to model transaction property and sequence, TGPT leverages a hybrid approach to encode MMTT data toward optimal information bandwidth between different data dimensions; 3) we explore how TGPT integrates LLM knowledge and compare TGPT with LLM-based baselines.

7.2 Transformer for Tabular and Temporal Data

Our transaction data’s MMTT format resembles the tabular, time series, and marked point process data. Many recent works have leveraged the Transformer to model the above data types. We review the representative works that motivated TGPT and compare their differences with our work.

TabNet [Arik and Pfister, 2021] and TabTransformer [Huang et al., 2020] motivate our metadata and feature Transformers design, especially the numerical and categorical field encoding. The tabular foundation models [Somepalli et al., 2021, He et al., 2023, Hollmann et al., 2025, Jingang et al., 2025] also inspire us to devise a foundation model for MMTT data. In contrast to most tabular foundation models, which only work on small-scale and low-dimensional tabular data, TGPT is validated on a more complex and large-scale MMTT data with multiple data modalities. Unlike recent foundation models targeting large-scale relational data [Fey et al., 2025, Ranjan et al., 2025], TGPT is designed around the sequential structure of payment transactions and tailored to Visa’s downstream tasks.

Most research on time series Transformers focus on better encoding of timestamps [Zerveas et al., 2021, Zhou et al., 2021, Garza et al., 2023, Liu et al., 2024, Ekambaram et al., 2024]. Those works motivate us in devising time embedding, temporal Transformer, and the self-supervised objective for time information. However, MMTT data is more complex than time series data: 1) MMTT has high-dimensional heterogeneous data fields for each timestamp, while time series data only has unimodal data type; 2) MMTT has irregular intervals between timestamps, while time series data has uniform intervals. Therefore, the above time series Transformers cannot be directly adopted in MMTT data.

Comparing to the time series data, the transaction sequence is more like a marked point process [Mei and Eisner, 2017] composed of discrete events with irregular intervals. Previous work use Transformer to model the point process’s intensity function [Zuo et al., 2020, Zhang et al., 2020, Yang et al., 2021] or the distribution of events and inter-event times [Panos, 2024]. Due to the multi-modal nature and multi-scale time info of the MMTT data, modeling it as a point process with Transformers will result in a fragmented model with excessive amount of parameters for each yet to be defined event type. Therefore, we resort to LLM’s next token prediction paradigm and using MSE loss for time prediction toward a unified and coherent Transformer architecture in TGPT.

Recent endeavors leverage Transformer-based LLMs to encode tabular [Ruan et al., 2024] and temporal [Zhang et al., 2024] data by translating the data into text. In this work, we do not adopt the above approach because: 1) predictive modeling does not require textual output, which is different from tabular QA and tabular understanding tasks with rich semantics; 2) MMTT data has high-dimensional metadata and features, and employing LLM to encode them is costly; 3) recent research [Tan et al., 2024, Van Breugel and Van Der Schaar, 2024] corroborates that LLMs do not have a significant advantage on many temporal forecasting and tabular classification tasks.

7.3 Continuous Space Tokenization and Feature Crossing

As detailed in the paper, the goal of designing the virtual token mechanism is to map continuous latent embeddings into a set of discrete tokens, enabling effective integration with another modality (such as metadata) or along another data dimension (such as temporal) while allowing flexible information bandwidth. A similar objective exists in other domains, such as computer vision and time series modeling. However, our realization of this objective employs a methodology resembling techniques developed for feature crossing.

Continuous Space Tokenization. Vision Transformer (ViT) [Dosovitskiy et al., 2020] splits an image into fixed-size patches, linearly projects each patch to an embedding, adds position encodings, and feeds the sequence to a Transformer. More recently, in vision-language models (VLMs) [Alayrac et al., 2022, Liu et al., 2023, Zhou et al., 2024], continuous latent embeddings from the image modality are tokenized into multiple patch embeddings and then fused into sequential language models. For tokenizing continuous spaces, pre-trained vector-quantized variational autoencoder (VQ-VAE) techniques are commonly applied [Van Den Oord et al., 2017, Razavi et al., 2019, Esser et al., 2021]. Time-LLM [Jin et al., 2023, 2024], as a representative Foundational time-series model [Liang et al., 2024], segments sequences into consecutive patches for tokenization and reprograms them with a small set of text prototypes that semantically describe each patch’s basic properties, resembling the role of the discrete codebook in VQ-VAE from another approach. In transaction modeling, however, pre-trained vector-quantization models are not readily available. Moreover, the latent representations of metadata, features, or transactions generally lack explicit physical semantics, unlike numerical time series that can be naturally described with terms such as “up” or “down”. Consequently, our approach aligns more closely with the ViT-style methodology, augmented with feature crossing.

Feature Crossing. Based on our extensive experience in transaction and feature modeling, feature crossing and interaction-based techniques have proven effective for extracting information from tabular data that contains diverse metadata fields and large numbers of numerical features [Xu et al., 2024]. Hence, instead of applying a simple linear projection, as adopted in ViT, in our approach, the VTL architecture employs a dual-channel design reminiscent of Wide & Deep [Cheng et al., 2016], DCN [Wang et al., 2017], and our prior TransNet design [Xu et al., 2024]. Unlike DCN, however, VTL does not explicitly perform complex feature crossing, as both preceding and subsequent Transformer blocks implicitly model these interactions. Our ablation study demonstrates the performance advantage of the dual-channel feature crossing design over the ViT-style linear projection (FMVTL-lin-map in Table 4).

8 Conclusion and Future Work

This work presents TransactionGPT (TGPT), a foundation model that captures complex consumer shopping dynamics from Multi-Modal-Temporal-Tabular (MMTT) data. Our novel 3D-Transformer architecture, augmented by a virtual token mechanism, is designed to effectively encode and fuse diverse data modalities. Extensive experiments on large-scale, real-world payment data validate TGPT’s ability to learn meaningful transaction patterns, leading to significant performance improvements on critical downstream tasks. Furthermore, we quantify the benefits of several designs that enhance the TGPT’s efficiency and scalability. This research opens a new avenue for foundation modeling of ubiquitous MMTT data on web. Future directions include accelerating model performance, developing superior multi-modal encoders, and exploring the joint optimization of MMTT foundation models with LLMs.

Core Contributors

Yingtong Dou, Zhimeng Jiang, Tianyi Zhang, Mingzhi Hu, Zhichao Xu, Yuzhong Chen

Contributors

Shubham Jain, Uday Singh Saini, Xiran Fan, Jiarui Sun, Menghai Pan, Junpeng Wang, Xin Dai, Liang Wang, Chin-Chia Michael Yeh, Yujie Fan, Yan Zheng, Vineeth Rakesh, Huiyuan Chen, Guanchu Wang, Mangesh Bendre, Zhongfang Zhuang, Xiaoting Li, Prince Aboagye, Vivian Lai, Minghua Xu, Hao Yang, Yiwei Cai, Mahashweta Das

Project Lead

Yuzhong Chen

References

- A. Aladangady, S. Aron-Dine, W. Dunn, L. Feiveson, P. Lengermann, and C. Sahm. From transactions data to economic statistics: Constructing real-time, high-frequency, geographic measures of consumer spending. In *Big Data for 21st Century Economic Statistics*. University of Chicago Press, 2019.
- J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- G. Aminian, A. Elliott, T. Li, T. C. H. Wong, V. C. Dehon, L. Szpruch, C. Maple, C. Read, M. Brown, G. Reinert, et al. Fraudtransformer: Time-aware gpt for transaction fraud detection. *arXiv preprint arXiv:2509.23712*, 2025.
- S. Ö. Arik and T. Pfister. Tabnet: Attentive interpretable tabular learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 6679–6687, 2021.
- D. Babaev, M. Savchenko, A. Tuzhilin, and D. Umerenkov. Et-rnn: Applying deep learning to credit loan applications. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2183–2190, 2019.
- A. C. Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten. Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications*, 51:134–142, 2016.
- A. Bazarova, M. Kovaleva, I. Kuleshov, E. Romanenkova, A. Stepikin, A. Yugay, D. Mollaev, I. Kireev, A. Savchenko, and A. Zaytsev. Universal representations for financial transactional data: embracing local, global, and external contexts. *arXiv e-prints*, pages arXiv–2404, 2024.
- I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- R. Bommasani. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- R. Brühl Gabrielsson et al. A foundation model for consumption, transactions, and actions: The inception of behaviorgpt. *Unbox AI Blog*, May 2025. URL <https://research.unboxai.com/foundation-model-for-consumption-transactions-and-actions.html>.
- H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, et al. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*, pages 7–10, 2016.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- V. Ekambaram, A. Jati, P. Dayama, S. Mukherjee, N. Nguyen, W. M. Gifford, C. Reddy, and J. Kalagnanam. Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. *Advances in Neural Information Processing Systems*, 37: 74147–74181, 2024.
- P. Esser, R. Rombach, and B. Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12873–12883, 2021.

- X. Fan, Z. Jiang, C.-C. M. Yeh, Y. Chen, Y. Dou, M. Pan, and Y. Zheng. Enhancing foundation models in transaction understanding with llm-based sentence embeddings. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 903–911, 2025.
- M. Fey, V. Kocijan, F. Lopez, J. E. Lenssen, and J. Leskovec. Kumorfm: A foundation model for in-context learning on relational data. 2025.
- A. Garza, C. Challu, and M. Mergenthaler-Canseco. Timegpt-1. *arXiv preprint arXiv:2310.03589*, 2023.
- S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on recommender systems*, pages 299–315, 2022.
- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- X. He, M. Zhou, M. Zhou, J. Xu, X. Lv, T. Li, Y. Shao, S. Han, Z. Yuan, and D. Zhang. Anameta: A table understanding dataset of field metadata knowledge shared by multi-dimensional data analysis tasks. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 9471–9492, 2023.
- S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag. Tabllm: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics*, pages 5549–5581. PMLR, 2023.
- N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637 (8045):319–326, 2025.
- E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin. Tabtransformer: Tabular data modeling using contextual embeddings. *arXiv preprint arXiv:2012.06678*, 2020.
- M. Javaheripi, S. Bubeck, M. Abdin, J. Aneja, S. Bubeck, C. C. T. Mendes, W. Chen, A. Del Giorno, R. Eldan, S. Gopi, et al. Phi-2: The surprising power of small language models. *Microsoft Research Blog*, 1(3):3, 2023.
- A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- T. Jiang, S. Huang, Z. Luan, D. Wang, and F. Zhuang. Scaling sentence embeddings with large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3182–3196, 2024.
- M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, et al. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728*, 2023.
- M. Jin, Y. Zhang, W. Chen, K. Zhang, Y. Liang, B. Yang, J. Wang, S. Pan, and Q. Wen. Position: What can large language models tell us about time series analysis. In *41st International Conference on Machine Learning*. MLResearchPress, 2024.
- Q. Jingang, D. Holzmüller, G. Varoquaux, and M. Le Morvan. Tabicl: A tabular foundation model for in-context learning on large data. In *Forty-second International Conference on Machine Learning*, 2025.

- W.-C. Kang and J. McAuley. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pages 197–206. IEEE, 2018.
- J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- G. Li, Y. Zhang, X. Chen, C. Li, B. Wang, L. Kong, W. Wang, W. Huang, and M. H. Y. Tan. Panther: Generative pretraining beyond language for sequential user behavior modeling. *arXiv preprint arXiv:2510.10102*, 2025.
- X. Li, Y. Li, X. Mo, H. Xiao, Y. Shen, and L. Chen. Diga: Guided diffusion model for graph recovery in anti-money laundering. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4404–4413, 2023.
- Y. Liang, H. Wen, Y. Nie, Y. Jiang, M. Jin, D. Song, S. Pan, and Q. Wen. Foundation models for time series analysis: A tutorial and survey. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pages 6555–6565, 2024.
- T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(2):318–327, 2018.
- C. Liu, L. Sun, X. Ao, J. Feng, Q. He, and H. Yang. Intention-aware heterogeneous graph attention networks for fraud transactions detection. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pages 3280–3288, 2021.
- H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long. itransformer: Inverted transformers are effective for time series forecasting. In *The Twelfth International Conference on Learning Representations*, 2024.
- L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- H. Mei and J. M. Eisner. The neural hawkes process: A neurally self-modulating multivariate point process. *Advances in neural information processing systems*, 30, 2017.
- M. Moor, O. Banerjee, Z. S. H. Abad, H. M. Krumholz, J. Leskovec, E. J. Topol, and P. Rajpurkar. Foundation models for generalist medical artificial intelligence. *Nature*, 616(7956):259–265, 2023.
- G. Moreira, S. Rabhi, J. Lee, R. AK, and E. Oldridge. Transformers4rec: Bridging the gap between nlp and sequential / session-based recommendation. In *RecSys*, 2021.
- I. Padhi, Y. Schiff, I. Melnyk, M. Rigotti, Y. Mroueh, P. Dognin, J. Ross, R. Nair, and E. Altman. Tabular transformers for modeling multivariate time series. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 3565–3569. IEEE, 2021.
- A. Panos. Decomposable transformer point processes. *Advances in Neural Information Processing Systems*, 37:88932–88955, 2024.
- O. Press and L. Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 157–163, 2017.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- R. Ranjan, V. Hudovernik, M. Znidar, C. Kanatsoulis, R. Upendra, M. Mohammadi, J. Meyer, T. Palczewski, C. Guestrin, and J. Leskovec. Relational transformer: Toward zero-shot foundation models for relational data. *arXiv preprint arXiv:2510.06377*, 2025.

- A. Razavi, A. Van den Oord, and O. Vinyals. Generating diverse high-fidelity images with vq-vae-2. *Advances in neural information processing systems*, 32, 2019.
- Y. Ruan, X. Lan, J. Ma, Y. Dong, K. He, and M. Feng. Language modeling on tabular data: A survey of foundations, techniques and evolution. *arXiv preprint arXiv:2408.10548*, 2024.
- H.-J. M. Shi, D. Mudigere, M. Naumov, and J. Yang. Compositional embeddings using complementary partitions for memory-efficient recommendation systems. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 165–175, 2020.
- P. Skalski, D. Sutton, S. Burrell, I. Perez, and J. Wong. Towards a foundation purchasing model: Pretrained generative autoregression on transaction sequences. In *ACM ICAIF*, pages 141–149, 2023.
- G. Somepalli, M. Goldblum, A. Schwarzschild, C. B. Bruss, and T. Goldstein. Saint: Improved neural networks for tabular data via row attention and contrastive pre-training. *arXiv preprint arXiv:2106.01342*, 2021.
- Stripe. Stripe accelerates the utility of ai and stablecoins with major launches. <https://stripe.com/newsroom/news/sessions-2025>, 2025. Accessed: 2025-06-13.
- M. Tan, M. Merrill, V. Gupta, T. Althoff, and T. Hartvigsen. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems*, 37:60162–60191, 2024.
- H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- B. Van Breugel and M. Van Der Schaar. Position: Why tabular foundation models should be a research priority. In *International Conference on Machine Learning*, pages 48976–48993. PMLR, 2024.
- A. Van Den Oord, O. Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Visa. Visa merchant data standards manual. <https://usa.visa.com/dam/VCOM/download/merchants/visa-merchant-data-standards-manual.pdf>, 2025. Accessed: 2025-06-13.
- R. Wang, B. Fu, G. Fu, and M. Wang. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD’17*, pages 1–7, 2017.
- Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun. Transformers in time series: a survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, pages 6778–6786, 2023.
- R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu. On layer normalization in the transformer architecture. In *International conference on machine learning*, pages 10524–10533. PMLR, 2020.
- M. Xu, H. Chen, M. Pan, Y. Chen, Y. Dou, and M. Das. Transnet: Transaction networks for tabular data. Technical Disclosure Commons, mar 2024. Available at: https://www.tdcommons.org/dpubs_series/6780.
- C. Yang, H. Mei, and J. Eisner. Transformer embeddings of irregularly spaced events and their participants. *arXiv preprint arXiv:2201.00044*, 2021.
- G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff. A transformer-based framework for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pages 2114–2124, 2021.

- J. Zhan, J. Dai, J. Ye, Y. Zhou, D. Zhang, Z. Liu, X. Zhang, R. Yuan, G. Zhang, L. Li, et al. Anygpt: Unified multimodal llm with discrete sequence modeling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9637–9662, 2024.
- D. Zhang, L. Wang, X. Dai, S. Jain, J. Wang, Y. Fan, C.-C. M. Yeh, Y. Zheng, Z. Zhuang, and W. Zhang. Fata-trans: Field and time-aware transformer for sequential tabular data. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 3247–3256, 2023.
- Q. Zhang, A. Lipani, O. Kirnap, and E. Yilmaz. Self-attentive hawkes process. In *International conference on machine learning*, pages 11183–11193. PMLR, 2020.
- X. Zhang, R. R. Chowdhury, R. K. Gupta, and J. Shang. Large language models for time series: A survey. *arXiv preprint arXiv:2402.01801*, 2024.
- Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, et al. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.
- C. Zhou, L. Yu, A. Babu, K. Tirumala, M. Yasunaga, L. Shamis, J. Kahn, X. Ma, L. Zettlemoyer, and O. Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. *arXiv preprint arXiv:2408.11039*, 2024.
- H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.
- S. Zuo, H. Jiang, Z. Li, T. Zhao, and H. Zha. Transformer hawkes process. In *International conference on machine learning*, pages 11692–11702. PMLR, 2020.

Appendix

A Compositional Embedding

Algorithm 1 shows the pseudocode of the compositional embedding class. In class initialization, a trainable `EmbeddingBag`¹ layer is created to store $m + 1$ embeddings with dimension = dim (Lines 2-5). k hash functions (e.g., `MurmurHash3`²) with different random seeds map the total number of entities n to a smaller shared embedding space of size m . The hashed indices are stored as non-trainable parameters in a lookup table, and the index 0 is reserved for unseen entities (Lines 6-14). During the forward pass (Lines 16-20), the input entity index e is mapped through the lookup table to obtain its hashed indices (Line 17), and the corresponding embeddings of hashed indices are aggregated using `EmbeddingBag` with mean pooling to obtain the output compositional embedding (Line 18).

Algorithm 1 Compositional Embedding Class.

```

1: procedure INIT( $n, m, dim, k$ )
2:   Initialize EmbeddingBag layer:
3:     Number of embeddings  $\leftarrow m + 1$ 
4:     Embedding dimension  $\leftarrow dim$ 
5:     Padding index  $\leftarrow 0$ 
6:   Initialize empty list  $hashed\_idx$ 
7:   for  $i \leftarrow 0$  to  $k - 1$  do
8:      $idx \in \mathbb{R}^{n+1} \leftarrow$  array of integers from 0 to  $n$ 
9:      $idx \leftarrow murmurhash3\_32(idx, seed = i)$ 
10:     $idx \leftarrow (idx \bmod m) + 1$ 
11:    Set  $idx[0] \leftarrow 0$  ▷ Padding index
12:    Append  $idx$  to  $hashed\_idx$ 
13:   end for
14:    $lookup\_table \in \mathbb{R}^{k \times (n+1)} \leftarrow$  stack arrays in  $hashed\_idx$ 
15: end procedure
16: procedure FORWARD( $e$ )
17:    $e\_indices \in \mathbb{R}^k \leftarrow lookup\_table$  indexed by  $e$ 
18:    $e\_embedding \in \mathbb{R}^m \leftarrow \mathbf{EmbeddingBag}(e\_indices)$ 
19:   return  $e\_embedding$ 
20: end procedure

```

B Proof of Theorem 1

Theorem 1. *If $1 < d_f \ll d_{\mathcal{M}}, 1 < v_f, v_t, |\mathcal{M}| \ll |\mathcal{F}|$, and $1 < w \ll \frac{d_{\mathbf{tr}}}{v_t}$, then $O_{3D-FMT} \ll O_{2D}$.*

Proof. Substituting the time complexities in $O_{3D-FMT} \ll O_{2D}$ by their definitions, we need to show:

$$\begin{aligned}
 & (|\mathcal{F}|d_f^2 + |\mathcal{F}|^2d_f)w + ((|\mathcal{M}| + v_f)d_{\mathcal{M}}^2 + (|\mathcal{M}| + v_f)^2d_{\mathcal{M}})w + v_tw \left(\frac{d_{\mathbf{tr}}}{v_t} \right)^2 + (v_tw)^2 \frac{d_{\mathbf{tr}}}{v_t} \\
 & \ll ((|\mathcal{M}| + |\mathcal{F}|)d_{\mathcal{M}}^2 + (|\mathcal{M}| + |\mathcal{F}|)^2d_{\mathcal{M}})w + wd_{\mathbf{tr}}^2 + w^2d_{\mathbf{tr}}
 \end{aligned}$$

We analyze the inequality by comparing the terms on the left-hand side (LHS) with those on the right-hand side (RHS).

Comparison of terms related to $TF(\mathbf{tr})$ and $TF(\mathcal{F})$:

- LHS terms: $(|\mathcal{F}|d_f^2 + |\mathcal{F}|^2d_f)w + ((|\mathcal{M}| + v_f)d_{\mathcal{M}}^2 + (|\mathcal{M}| + v_f)^2d_{\mathcal{M}})w$

¹<https://docs.pytorch.org/docs/stable/generated/torch.nn.EmbeddingBag.html>

²https://scikit-learn.org/stable/modules/generated/sklearn.utils.murmurhash3_32.html

- RHS terms: $((|\mathcal{M}| + |\mathcal{F}|)d_{\mathcal{M}}^2 + (|\mathcal{M}| + |\mathcal{F}|)^2d_{\mathcal{M}})w$

Given the conditions $|\mathcal{M}| \ll |\mathcal{F}|$ and $v_f \ll |\mathcal{F}|$, we have the asymptotic approximations $(|\mathcal{M}| + |\mathcal{F}|) \approx |\mathcal{F}|$ and $(|\mathcal{M}| + v_f) \ll |\mathcal{F}|$. The dominant part of the RHS is therefore approximately $|\mathcal{F}|d_{\mathcal{M}}^2 + |\mathcal{F}|^2d_{\mathcal{M}}$. Now we show that each component of the LHS is asymptotically smaller than the corresponding dominant component of the RHS:

- Since $d_f \ll d_{\mathcal{M}}$, we have $|\mathcal{F}|d_f^2 \ll |\mathcal{F}|d_{\mathcal{M}}^2$ and $|\mathcal{F}|^2d_f \ll |\mathcal{F}|^2d_{\mathcal{M}}$.
- Since $(|\mathcal{M}| + v_f) \ll |\mathcal{F}|$, we have $(|\mathcal{M}| + v_f)d_{\mathcal{M}}^2 \ll |\mathcal{F}|d_{\mathcal{M}}^2$ and $(|\mathcal{M}| + v_f)^2d_{\mathcal{M}} \ll |\mathcal{F}|^2d_{\mathcal{M}}$.

As every term on the LHS is asymptotically negligible compared to a corresponding term on the RHS, their sum is also asymptotically negligible.

Comparison of terms related to $\text{TF}(\mathcal{S})$:

- LHS terms: $\alpha_1 = v_t w \left(\frac{d_{\text{tr}}}{v_t} \right)^2 + (v_t w)^2 \frac{d_{\text{tr}}}{v_t} = \frac{w d_{\text{tr}}^2}{v_t} + v_t w^2 d_{\text{tr}}$
- RHS terms: $\alpha_2 = w d_{\text{tr}}^2 + w^2 d_{\text{tr}}$

To prove $\alpha_1 \ll \alpha_2$, we analyze their ratio $R = \frac{\alpha_1}{\alpha_2}$:

$$R = \frac{\frac{w d_{\text{tr}}^2}{v_t} + v_t w^2 d_{\text{tr}}}{w d_{\text{tr}}^2 + w^2 d_{\text{tr}}} = \frac{d_{\text{tr}}^2/v_t + v_t w d_{\text{tr}}}{d_{\text{tr}}^2 + w d_{\text{tr}}} = \frac{d_{\text{tr}}/v_t + v_t w}{d_{\text{tr}} + w}$$

Let's factor out d_{tr} from the numerator and denominator:

$$R = \frac{\frac{d_{\text{tr}}}{v_t} \left(1 + \frac{v_t^2 w}{d_{\text{tr}}} \right)}{d_{\text{tr}} \left(1 + \frac{w}{d_{\text{tr}}} \right)} = \frac{1}{v_t} \cdot \frac{1 + \frac{v_t^2 w}{d_{\text{tr}}}}{1 + \frac{w}{d_{\text{tr}}}}$$

As $w \ll \frac{d_{\text{tr}}}{v_t}$, both $\frac{v_t^2 w}{d_{\text{tr}}}$ and $\frac{w}{d_{\text{tr}}}$ approach 0. Therefore, the ratio R asymptotically approaches $\frac{1}{v_t}$:

$$\lim R = \frac{1}{v_t}$$

Given the condition $v_t > 1$, we have $1/v_t < 1$. This shows that α_1 is asymptotically a fraction of α_2 , representing a significant reduction. Since the time complexity terms on the LHS are asymptotically smaller than their counterparts on the RHS, the sum on the LHS is asymptotically smaller than the sum on the RHS. Thus, we have shown that $O_{3\text{D-FMT}} \ll O_{2\text{D}}$. $\square$

C LLM-based Entity Embedding Initialization

Among all entities in the consumer payment ecosystem, merchant category code (MCC) is particularly semantic rich thanks to the ISO 18245³ defining the meaning and description of each MCC code. According to the Visa Merchant Data Standards Manual [Visa, 2025], we use the extended description of an MCC to prompt LLMs to generate the MCC embeddings. Motivated by the one-word prompt design for sentence embedding [Jiang et al., 2024], we design the following prompt template:

The MCC **{mcc}**, titled **{title}**, is described as follows: **{description}**. This category includes **{included}**, similar MCCs are **{similar}**. Please provide the embedding of the MCC **{mcc}** in the next token.

The **{included}** is an extended list of merchant types belonging to the corresponding MCC. The **{similar}** is a list of MCCs similar to the current MCC. Those fields can be found in the Visa Merchant Data Standards Manual mentioned above.

³<https://www.iso.org/standard/79450.html>

By prompting the open-weights LLMs using the above prompt template, we extract the activations of an LLM’s last layer as the embedding of the input MCC. As the last layer’s activations have included the context information and summarized the MCC information. This solution is cost effective and scalable since no fine-tuning is required, and it is faster than sentence embedding models with bi-directional attentions, which makes it is possible to scale to high-cardinality entities. Its LLM-agnostic design also enables us to adopt the latest open-weights LLMs and LLMs with various sizes. Please find more details of applying sentence embeddings to foundation models in our extended work [Fan et al., 2025].

D LLM for Transaction Field Prediction

In Sec. 5.3 and Fig. 6, we compare TGPT performance on next transaction MCC prediction against LLM-based methods. We present the technical details of LLM-based methods, which can be generalized to predicting any field of future transactions.

Assuming the future transaction is dependent on historical transactions, we design the following text template for LLM fine-tuning:

[More historical transaction info...]
 Target field: **{field_value}**, transaction amount: **{amount}**, transaction date: **{date}**, ...,
 transaction time: **{time}**. ▷ last transaction info.
 Target field: [MASK]. ▷ field to be predicted.

Specifically, the descriptions of the key metadata of each historical transaction are used as the context. One can flexibly adjust the number historical transactions and which transaction information to be described in the context. The target field value to be predicted for the future transaction is the last token of the text template, which is masked during training.

In the MCC prediction task, since the target field value (i.e., four-digit MCC codes) are not in the vocabulary of open-weights LLMs, we insert 800 new special tokens to each LLM’s vocabulary to represent MCC codes. Consequently, fine-tuning LLMs on this data is required to let LLMs understand transaction patterns and learn the correlation between historical transaction information and future field value represented by special tokens.

Although expanding a pretrained LLM’s vocabulary will add overhead during fine-tuning, this approach is common in multi-modal LLM [Zhan et al., 2024] and generative recommendation [Geng et al., 2022], and it facilitates model evaluation. In contrast, using each MCC’s plain-text name has following challenges: 1) each MCC’s name includes 1 to 30 tokens (e.g., **MCC 4011**: Railroads and **MCC 4814**: Telecommunication Services, including Local and Long-Distance Calls, Credit Card Calls, Calls Through Use of Magnetic Stripe-Reading Telephones, and Fax Services); 2) some MCCs share similar words in their names (e.g., **MCC 5532**: Automotive Tire Stores and **MCC 5533**: Automotive Parts and Accessories Stores). It is non-trivial to extract the exact MCC entity from a sequence of generated tokens.

In a nutshell, the LLM’s fine-tuning and inference process is a next token prediction task with only one forward pass (i.e., only predicting the next one token). In the experiment, we fine-tune three open-weights LLMs (Llama2-7B [Touvron et al., 2023], Mistral-7B-v0.1 [Jiang et al., 2023], Phi2-2.7B [Jawaheripi et al., 2023]) with different sizes and from different model families. LoRA [Hu et al., 2022] and FSDP [Zhao et al., 2023] are used to accelerate fine-tuning and improve scalability.

E Practical Guide on Metadata Transformer Output Integration Strategies

Table 7 provides an empirical comparative summary of the strengths and limitations associated with the five integration strategies discussed in Sec 6: concatenation, MLP compression (or linear mapping), fooling (summation/average/max), field-embedding selection + pooling, and field-embedding selection + concatenation.

Table 7: Empirical comparison of Metadata Transformer output integration strategies in TGPT-2D. The method adopted for results in Table 4 is highlighted.

Method	Bandwidth	Noise Level	OOM Risk	Information Loss	Performance
Concat	High	High (no filtering)	High	Low	Impractical due to memory limits
MLP compression	Flexible (adjustable via projection)	Moderate	Moderate	Moderate (compression removes detail)	Prone to overfitting; sub-optimal in practice
Pooling (sum/avg/max)	Low (same as single field embedding)	High (aggregates all fields)	Low	High (pooling discards information)	Weak — signal diluted and bandwidth-limited
Field selection + pooling	Low (same as single field embedding)	Lower than full pooling	Low	High (field omission + pooling)	Better than full pooling, but bandwidth-limited
Field selection + concat	Moderate-high (depends on #fields)	Low (noise filtered)	Moderate	Moderate (field omission)	Balance of stability, efficiency, and performance

Table 8: The search space of key model components.

Component	Search Space
# Transformer layers	[1,2,3,4]
# Attention heads	[2,4,6,8]
Metadata embedding dimension	[4,8,16,32,64,128,256]
Feature embedding dimension	[4,8,16,32,64,128]
# Virtual tokens	[1,2,4,8,16,32]